OWASP
Smart Contract Security

scs.owasp.org ↗

Security Handbook :

SDK Security Testing

08

Security Handbook Series

GRANT SUPPORT • 2025-2026

ecosystem
support
program

Supported by Ethereum Foundation's
Ecosystem Support Program (ESP)

Contents

Frontispiece ... 3
- About This Handbook ... 3
- Copyright and License ... 4
- Author and Creative Credits ... 4
- Purpose and Scope ... 5
- Target Audience ... 5
- How to Use This Handbook ... 5
- Relationship to SCSVS, SCSTG, and SCWE ... 6

Part I: Foundations ... 7
- 1. Introduction to SDK Security ... 7
- 2. Threat Model for SDKs ... 10

Part II: Testing Methodology ... 19
- 3. Testing Framework Overview ... 19
- 4. Supply Chain and Dependency Testing ... 24
- 5. API and Interface Security ... 33
- 6. Wallet and Key-Handling SDKs ... 37
- 7. Contract Interaction SDKs ... 41
- 8. Front-End and Mobile SDKs ... 44

Part III: Implementation and Operations ... 47
- 9. Integrating SDK Testing into CI/CD ... 47
- 10. Selecting and Evaluating Third-Party SDKs ... 52
- 11. Manual Review and External Security Review Preparation ... 54
- 12. Responsible Disclosure for SDK Vulnerabilities ... 57

Part IV: Appendices ... 60
- 13. Appendix A: Glossary ... 60
- 14. Appendix B: SDK Security Testing Checklist ... 65
- 15. Appendix C: References and Further Reading ... 69

Appendix C: References and Further Reading ... 71
- Standards and Frameworks ... 71
- Incidents and Case Studies ... 71
- Tools and Documentation ... 72
- Further Reading ... 72

Frontispiece

OWASP SMART CONTRACT SECURITY PROJECT

SDK Security Testing

About This Handbook

SDK Security Testing is part of the OWASP Smart Contract Security Handbook Series. The series extends smart contract security beyond contract code into the people, process, application, infrastructure, and operational layers surrounding Web3 systems.

Each handbook is designed as a defensive field reference for architects, engineers, security practitioners, incident responders, auditors, and organizational leaders. It complements the OWASP Smart Contract Security Verification Standard (SCSVS), Smart Contract Security Testing Guide (SCSTG), Smart Contract Weakness Enumeration (SCWE), Smart Contract Top 10, and SCS Checklist without replacing those resources.

SERIES EDITION **2026**

Copyright and License

Author and Creative Credits

PROJECT LEAD

Shashank | CredShields

CEO and Co-Founder

AUTHOR | PROJECT MAINTAINER

Pratik Lagaskar

Security Researcher

COVER PAGE DESIGNS

Siddharth Neekher

Lead UI Designer @ CredShields

OWASP Smart Contract Security Project

Purpose and Scope

This handbook covers testing and security assessment methodology for software development kits (SDKs) used across Web3 and smart contract ecosystems: wallet SDKs, contract-interaction SDKs, front-end SDKs, and mobile SDKs. It aligns with the software supply chain standards that govern how these components are built, packaged, and distributed: Supply-chain Levels for Software Artifacts (SLSA) v1.2 Build and Source tracks, Software Bill of Materials (SBOM) formats CycloneDX and SPDX, and Sigstore for signed provenance. It draws on OWASP's own supply chain guidance, including Top 10 category A03 Software Supply Chain Failures, and on the dependency-tampering patterns documented across the wider OWASP ecosystem. Threat modeling, test design, and verification steps are aligned with the Smart Contract Security Verification Standard (SCSVS) and the Smart Contract Security Testing Guide (SCSTG).

An SDK sits directly between a developer's application code and the wallet, contract, or network it talks to, so a single compromised release can silently redirect signing requests or exfiltrate keys across every downstream application that imports it. The Ledger Connect Kit compromise of December 2023 made this concrete: a phishing attack against a former employee let attackers push a malicious update through the SDK's own CDN-hosted bundle, and dozens of dApps that trusted the package without pinning or verifying it served the malicious code to their users. This handbook treats SDK selection, testing, and ongoing monitoring as a discipline distinct from application-level contract auditing, because the vulnerability class runs someone else's code, built by someone else's pipeline, inside your application's trust boundary.

Target Audience

Developers, security engineers, and QA practitioners responsible for selecting, integrating, or building SDKs used in dApps and smart contract tooling.

How to Use This Handbook

Read Part I first to establish the threat model: the trust boundaries an SDK crosses, the actors who can compromise it, and how SLSA frames supply chain risk. Part II is the testing reference, organized by concern (supply chain and dependency testing, API and interface security) and then by SDK type (wallet, contract-interaction, front-end and mobile), each closing with a test-case checklist you can run directly against a candidate SDK. Part III moves from testing to process: wiring these checks into CI/CD, evaluating a third-party SDK before adoption, preparing an SDK

for external review, and handling a disclosed vulnerability responsibly. The appendices hold a glossary, a consolidated SDK security testing checklist, and the full bibliography. A team vetting a new wallet SDK before integration can go straight to Chapter 6 and Appendix B.

Relationship to SCSVS, SCSTG, and SCWE

This handbook applies the SCS standards to a component the standards themselves treat as a black box: code your team did not write but ships inside a trust boundary your team owns. SCSVS defines the verification requirements an application must satisfy; the SDK test cases and checklists here are how you confirm a third-party dependency does not undermine those requirements before you ship it. SCSTG describes testing methodology for smart contract systems generally; Part II extends that methodology outward to the SDK layer, covering static and dynamic analysis, composition and dependency analysis, and behavioral testing of code outside the audited contract surface. The Smart Contract Weakness Enumeration (SCWE) catalogs weaknesses in contract code; this handbook fills the adjacent gap at the SDK and dependency boundary, where the weakness lives in a package.json entry rather than a Solidity file. The CDN and Front-End Supply Chain Security Handbook (01) shares this handbook's SLSA, SBOM, and Sigstore foundations from the delivery-infrastructure side, and the Incident Response Handbook (06) owns the response process that Chapter 12's responsible disclosure guidance feeds into.

Part I: Foundations

This part fixes the vocabulary this handbook uses throughout: what an software development kit (SDK) is in a Web3 context, the four types this handbook treats separately, and the trust boundaries an SDK crosses between application code, a wallet, and a network. It then builds the threat model, actor by actor and threat by threat, that every later testing chapter assumes the reader already has in mind.

1. Introduction to SDK Security

1.1 Why SDK Security Matters in Web3

A software development kit (SDK) is a packaged set of libraries, tools, and interfaces that lets an application talk to a specific system, in Web3's case a wallet, a smart contract, or a blockchain node, without the application team writing that integration from scratch. `ethers.js`, `viem`, the WalletConnect SDK, and Solana's `@solana/web3.js` are all SDKs in this sense: a developer imports one, and from that point forward the SDK's code, not the developer's own, constructs transactions, manages remote calls, or brokers the signing request. That delegation is the entire point of an SDK, and it is also the entire risk. Application code decides *what* to do; SDK code decides *how* the bytes that leave the browser or backend are actually shaped, and neither a wallet nor a smart contract has any way to distinguish a legitimate SDK call from a compromised one.

The blast radius of an SDK compromise dwarfs a typical application bug, because a single publish reaches every downstream consumer at once. The Ledger Connect Kit compromise of December 2023 is the reference case for this handbook: a phishing attack against a former Ledger employee's npm account let an attacker push a malicious update to a widely embedded wallet-connection SDK, and every dApp that pulled the update without pinning or verifying it served the tampered bundle to its own users, draining funds from wallets that never interacted with Ledger's own infrastructure directly (Ledger's incident letter). Nearly two years later, the same pattern repeated at even greater scale. In September 2025, attackers phished the maintainer of `chalk`, `debug`, and sixteen other foundational npm packages with a combined two billion weekly downloads, then injected code that hooks `window.ethereum` and rewrites transaction payloads in

the browser (Socket, Aikido). Neither `chalk` nor `debug` is itself a wallet SDK; both are transitive dependencies pulled in by SDKs that are. Socket's 2025 tracking found that roughly three-quarters of the blockchain-related malicious packages it identified were published to npm (OWASP Web3 Attack Vectors Top 15), the registry nearly every Web3 front end and Node-based backend depends on somewhere in its dependency graph.

1.2 Types of SDKs (Wallet, Contract, Front-End, Mobile)

SDKs used across a Web3 stack differ enough in what they touch that a single control set does not fit all of them. Four categories cover the practical landscape this handbook tests against.

Wallet SDKs manage or broker access to private keys and signing: WalletConnect, the Ledger and Trezor connector libraries, MetaMask's SDK, and Coinbase Wallet SDK. These sit closest to the money; a compromise here can sign or approve a transaction directly, or silently substitute the recipient address a user believes they are approving.

Contract-interaction SDKs encode function calls, manage application binary interfaces (ABIs), and submit transactions to a network through remote procedure call (RPC) nodes: `ethers.js`, `viem`, `web3.js`, and Solana's `@solana/web3.js` and Anchor client libraries. They rarely hold keys themselves, but they build the exact bytes a wallet blindly signs, so a corrupted encoding step, a swapped parameter order, a rewritten recipient, produces a technically valid, cryptographically sound transaction that does the wrong thing, with no error for the user to catch.

Front-end SDKs are UI and integration libraries bundled into the browser application: RainbowKit, wagmi hooks, and chain-abstraction toolkits. They overlap with the delivery-layer concerns of the CDN and Front-End Supply Chain Security Handbook (01), but the concern here is the package itself, not the CDN that later serves it.

Mobile SDKs ship inside native iOS and Android apps, including wallet apps, and carry risks specific to app-store distribution: a compiler-level compromise, Xcode itself was weaponized in the XcodeGhost campaign, which silently injected malicious code into thousands of iOS apps compiled with a tampered toolchain downloaded from unofficial mirrors, or a legitimate-looking ad or analytics SDK that turns out to exfiltrate data, as with the Android.Spy.SpinOk module found embedded in 101 apps with over 421 million combined installs before it was cleaned up (Doctor Web).

SDK Type	Representative Examples	What It Touches	Blast Radius if Compromised
Wallet	WalletConnect, MetaMask SDK, Ledger Connect Kit	Signing requests, key material, approval UI	Direct fund theft, silent approval hijack
Contract-interaction	ethers.js, viem, web3.js, Anchor client	ABI encoding, transaction construction, RPC submission	Valid-looking transactions with corrupted payloads

SDK Type	Representative Examples	What It Touches	Blast Radius if Compromised
Front-end	wagmi, RainbowKit, chain-abstraction kits	Browser bundle, UI state, wallet connection flow	Phishing UI injection, connection hijack
Mobile	Wallet-app SDKs, ad/analytics SDKs, compiler toolchains	Device storage, clipboard, app binary at compile time	Bulk credential/seed exfiltration across app-store installs

1.3 Relationship to SCSVS, SCSTG, SCWE

The OWASP smart contract standards were built around the assumption that the code under review is the code the team wrote. SDKs break that assumption: the vulnerability lives in a `package.json` entry, not a Solidity file, and this handbook exists to extend each standard to that boundary. The Smart Contract Security Verification Standard (SCSVS) states verifiable requirements an application must satisfy across categories such as access control, cryptography, and communications; the SDK test cases in Part 2 are how a team confirms a third-party dependency does not quietly undermine those requirements before it ships inside the application's trust boundary. The Smart Contract Security Testing Guide (SCSTG) documents testing methodology for contract systems; Part 2 extends that methodology outward into static analysis, dependency composition analysis, and behavioral testing of SDK code that a contract-focused guide never reaches. The Smart Contract Weakness Enumeration (SCWE) catalogs weaknesses in contract code; the SDK and dependency layer is a class of weakness the enumeration does not cover, and this handbook fills that gap.

The OWASP Web3 Attack Vectors Top 15 frames three of its fifteen categories directly around this handbook's scope: WA02 (supply chain attacks against npm, PyPI, and other open-source registries), WA14 (wallet software, extension, and app compromises), and WA15 (nation-state infiltration through fake hiring and malicious open-source contributions). Companion handbooks own the adjacent pieces of this picture: Handbook 01 covers the CDN and front-end delivery layer that hosts an SDK's bundled output, Handbook 03 (Employee Lifecycle Security) covers the insider and fake-hire vectors behind WA15, Handbook 06 (Incident Response) owns the response process that this handbook's disclosure guidance in Chapter 12 feeds into, Handbook 07 (Infrastructure Security) covers the CI/CD and registry infrastructure an SDK's own build depends on, and Handbook 10 (Web3 Attack Vectors Mapping) maps the full Top 15 in detail.

1.4 How to Use This Handbook

Read this Part once to fix the vocabulary: what an SDK is, the four types treated separately, and the trust-boundary and threat model every later chapter assumes. After that, treat the handbook as a reference shelf rather than a cover-to-cover read. Testing a candidate wallet SDK before integration starts at Chapter 6. Wiring dependency scanning into a pipeline is Chapter 9. Preparing an SDK a team maintains for external review is Chapter 11. Handling a disclosed vulnerability in a dependency already shipped is Chapter 12.

Part 2 (Chapters 3 through 8) is the testing methodology core, organized first by concern, supply chain and dependency testing in Chapter 4, application programming interface (API) and interface security in Chapter 5, and then by SDK type across Chapters 6 through 8, each closing with a checklist meant to be run directly against a candidate package rather than read and set aside. Part 3 (Chapters 9 through 12) moves from testing to process: CI/CD integration, third-party SDK evaluation before adoption, review preparation, and responsible disclosure. Part 4 holds the glossary, a consolidated checklist that pulls every chapter's individual list into one document, and the full bibliography. Cross-references throughout point to sibling handbooks, 01, 03, 06, 07, and 10, wherever a topic crosses into delivery infrastructure, personnel risk, incident response, or infrastructure hardening rather than the SDK itself.

2. Threat Model for SDKs

2.1 Trust Boundaries and Data Flow

A trust boundary marks the point where data crosses from one level of assumed trust to another, and a useful threat model requires the boundaries to be drawn before a single vulnerability class is considered. An SDK crosses four such boundaries in a typical Web3 application, and treating any one of them as trusted by default is where nearly every incident in this Part begins.

The first boundary is the **import boundary**: application code trusts that the SDK package declared in a manifest file is the code that actually gets installed, at the version and with the behavior a developer reviewed. The second is the **update boundary**: because most projects declare a version range rather than an exact pin, the application implicitly re-extends that same trust to every future patch release the registry serves, without a human reviewing it again. The third is the **signing boundary**: a wallet trusts that the transaction payload an SDK hands it for signature reflects the user's actual intent, with no independent way to verify that the SDK's encoding step did not substitute a value along the way. The fourth is the **network boundary**: the SDK trusts data returned by an RPC node or API endpoint, balances, gas estimates, contract addresses, and frequently passes that data back to the application or wallet without re-validation.

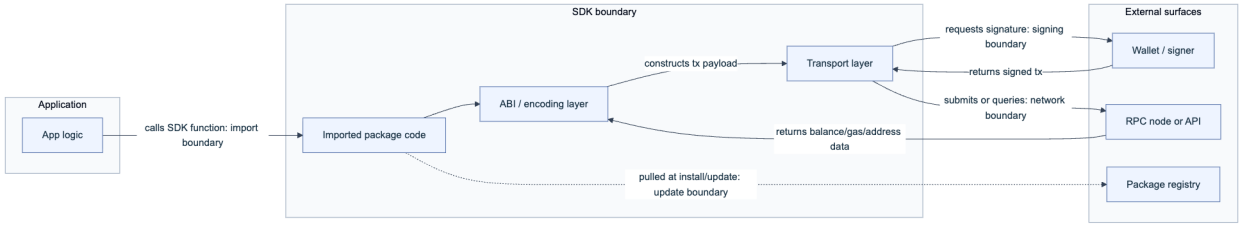

Figure 1. The four trust boundaries an SDK crosses. Each arrow is a point where a defender should ask what happens if the data crossing it is wrong.

2.2 Threat Actors (Malicious Package, Compromised Dependency, User Error)

Not every SDK-layer incident has the same actor behind it, and the categories worth separating call for different controls.

A **malicious package publisher** sets out from the start to deceive: a typosquatted name (`web3js` instead of `web3`), a convincing but fake fork of a popular library, or a brand-new package designed to look like tooling a developer would plausibly need. MITRE ATT&CK's (Adversarial Tactics, Techniques, and Common Knowledge) T1195 Supply Chain Compromise sub-technique T1195.001, Compromise Software Dependencies and Development Tools, catalogs this pattern generically across platforms. MITRE also maintains a sibling framework, AADAPT (Adversarial Actions in Digital Asset Payment Technologies), which applies the same tactic-and-technique structure specifically to blockchain and digital-asset systems and is worth cross-referencing alongside ATT&CK when threat-modeling a wallet SDK.

Figure. A typosquatting example in a browser address bar: a misspelled destination that looks close enough to the intended one to fool a quick glance. The same technique drives typosquatted package names such as `web3js` in place of `web3`. Source: Wikimedia Commons, CC0 1.0 Universal Public Domain Dedication.

A **compromised dependency maintainer** is a legitimate, previously trustworthy account taken over by an attacker, usually through credential phishing rather than a code-level exploit. The September 2025 compromise of the `chalk` and `debug` maintainer began with an email spoofing npm's own domain, warning of an imminent account lockout over outdated authentication and linking to a credential-harvesting page (Aikido). The 2018 `event-stream` incident took a slower route: an attacker earned co-maintainer status on a legitimate, popular package through ordinary open-source contribution, then added an encrypted payload as a new dependency, `flatmap-stream`, that activated only when it detected the Copay bitcoin wallet in the surrounding dependency tree (GitHub issue #116).

User error is the quieter category, and the one entirely within a development team's own control: installing a package without checking its download history, ignoring a lockfile and letting a floating version range pull an unreviewed patch, or granting a wallet-connected dApp broader signing scope than an interaction requires. None of the incidents above required a defender mistake to succeed, but most were survivable by a team that pinned versions and reviewed diffs before upgrading, which is why Chapter 4 treats dependency hygiene as a control, not an afterthought.

Actor	Access Path	Objective	Representative Incident
Malicious package publisher	Typosquat, fake fork, or new deceptive package	Immediate credential or key theft on install or first run	Ongoing typosquat campaigns (WA02)
Compromised dependency maintainer	Phished credentials or long-term social engineering into commit access	Wallet-draining code shipped in a trusted, high-download package	`chalk` / `debug` (Sept 2025); `event-stream` (2018)
Nation-state actor	Fake hiring, long-horizon open-source contribution, targeted CI compromise	Espionage, sanctioned-state revenue generation	WA15 patterns; long-horizon maintainer trust-building
User/developer error	Unpinned versions, disabled lockfiles, over-scoped wallet permissions	Not intentional; amplifies every row above	Present in nearly every major incident's spread

2.3 Supply Chain and Dependency Tampering (Build, Registry, Dependency Updates)

Dependency tampering happens at three distinct points, and each has a different defensive answer.

Build-time tampering alters what a package does during its own installation or compilation, before any application code runs. An npm `postinstall` script, a Python package's `setup.py`, or a similar lifecycle hook executes arbitrary code the moment a package manager installs it, with no import statement required. A defender reviewing a new dependency should treat any lifecycle script as a first-class thing to read, not skip:

```
{
  "scripts": {
    "postinstall": "node ./scripts/postinstall.js"
  }
}
```

That single line is not itself malicious, but it is the exact mechanism class the tj-actions/changed-files compromise (CVE-2025-30066) used at the continuous integration and continuous deployment (CI/CD) level: a widely used GitHub Action, invoked by more than 23,000 repositories, had its version tag repointed to a commit that dumped runner memory into build logs, exposing secrets to anyone who could read the log.

Registry-time tampering compromises the account or infrastructure that publishes a package, so the malicious code never touches the project's source repository at all, only the artifact the registry actually serves. This is the pattern behind both the `event-stream` and `chalk`/`debug` incidents from Section 2.2: the package a build tool pulled differed from what a reviewer would have found by reading the project's public source repository.

Dependency-update tampering exploits how most projects consume dependencies: a caret or tilde version range (`^5.3.0`) rather than an exact pin, which lets a CI pipeline or a fresh install silently pull a newer, malicious patch release with no code review at all. A related but distinct vector is dependency confusion, where an attacker publishes a public package sharing the name of an organization's private internal package; because public registries are often resolved ahead of private ones by default, the build silently resolves to the attacker's version. Alex Birsan's original 2021 disclosure demonstrated this technique against more than 35 companies, including Apple, Microsoft, and PayPal, and drew bounty payouts up to $40,000 per company.

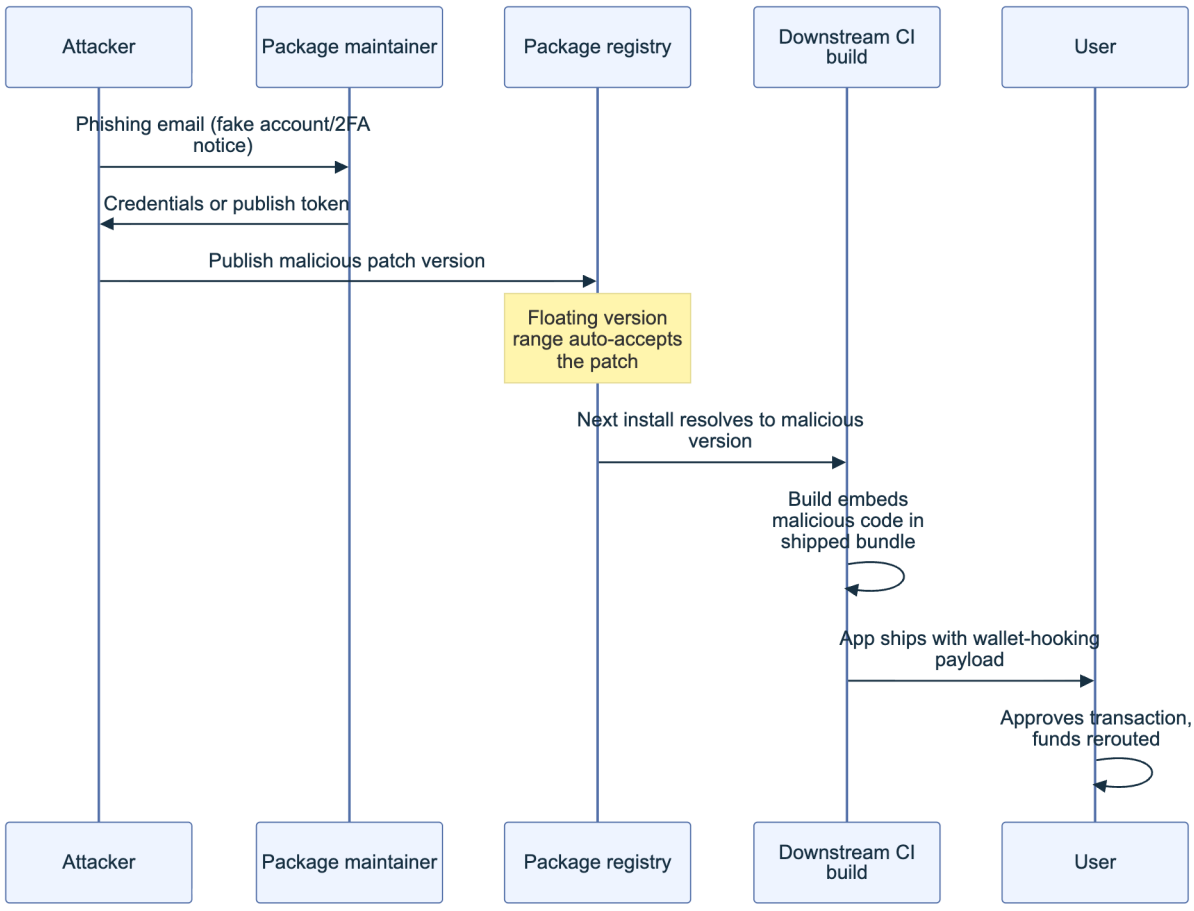

Figure 2. A registry-level compromise propagates to every downstream consumer on the next install, with no code review in the path.

2.4 STRIDE and Attack Scenarios for SDKs

Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege (STRIDE) applied to the SDK layer surfaces scenarios a generic application threat model tends to miss, because the vulnerable component is neither the front end nor the contract: it is the code connecting them.

STRIDE Category	SDK Scenario	Primary Control
Spoofing	Typosquatted package name (`ehters.js`, `web3-js`) mimics a real SDK in search results and install commands	Download-count and publisher verification, lockfile pinning
Tampering	A legitimate, previously clean package version is republished with injected code	Hash pinning, signed provenance verification (Chapter 6)
Repudiation	No signed record of who published a given version, so a malicious release cannot be attributed after the fact	Signed commits, provenance attestations, transparency logs
Information disclosure	SDK telemetry, error reporting, or a build-time script exfiltrates keys, seed phrases, or environment secrets	Network egress review, secret scanning, disabling unneeded telemetry
Denial of service	SDK hardcodes or silently defaults to a single RPC endpoint that becomes unavailable or rate-limits under load	Provider redundancy, configurable endpoints, circuit breakers
Elevation of privilege	SDK requests broader wallet permissions than the interaction needs (a blanket signing method instead of a scoped one)	Least-privilege signing method choice, permission review (Chapter 6)

A worked scenario ties the row to the earlier sections. A contract-interaction SDK adds a convenience method that silently falls back to a public, unauthenticated RPC endpoint whenever a developer's configured provider times out. Spoofing risk: nothing in a typical dependency review catches that fallback endpoint being swapped for a malicious one in a later release. Tampering risk: an attacker controlling that fallback endpoint returns a manipulated token balance or gas estimate to the application. Elevation risk: because the SDK, not the application, decides which endpoint to trust, the application has no visibility into the substitution at all. The controls that close every branch here are the same ones the rest of this Part builds toward: pin exact versions, verify provenance before upgrading, and never let an SDK make a trust decision, which endpoint, which permission scope, that the application cannot see or override.

2.5 Supply Chain and Distribution Risks

Section 2.3 covered how a single package gets tampered with. This section covers the infrastructure that distributes packages at scale, where a single point of failure reaches every consumer simultaneously regardless of how carefully any one team reviews its own dependencies.

Registry account security is the weakest link most often exploited, because a maintainer with commit access to a package carrying hundreds of millions of weekly downloads is a target disproportionate to any protection most individual volunteers have in place. The phishing email used against the `chalk` / `debug` maintainer in September 2025 spoofed npm's own domain and exploited exactly this asymmetry: one successful phish against one volunteer maintainer propagated to every project that had that package anywhere in its dependency tree (Aikido). Registry infrastructure itself is a second layer of risk: npm, PyPI, and mobile app stores each run their own vetting and takedown processes, and response time varies widely. The SpinOk Android SDK spyware remained distributed inside legitimate apps on Google Play for an extended period after independent researchers first flagged it, reaching over 421 million cumulative installs before a cleaned-up version shipped (Doctor Web).

Web3 tooling specifically skews toward one registry, which concentrates the risk further. Socket's 2025 tracking found that roughly three-quarters of the blockchain-related malicious packages it identified were published to npm (OWASP Web3 Attack Vectors Top 15, WA02), reflecting how much of the Web3 front-end and tooling ecosystem sits on the JavaScript package graph regardless of which chain a project targets. Distribution risk also overlaps with the CDN layer covered in Handbook 01: an SDK's bundled output is frequently served from the same third-party CDN infrastructure as any other front-end asset, so a CDN compromise and an SDK registry compromise can produce the identical user-facing failure, a tampered script running in a browser, by two entirely different supply-chain paths.

2.6 SLSA Threat Model and Mitigations (slsa.dev)

SLSA (Supply-chain Levels for Software Artifacts) is the cross-industry framework for describing, in verifiable and incremental terms, how confident a consumer can be that a software artifact was built the way its publisher claims. Originally released as v1.0, the specification has since progressed to v1.2, and v1.0 is now formally retired in its favor; the core structure both share is a Build Track describing how trustworthy an artifact's provenance is, joined in the current version by a Source Track describing how trustworthy the source repository itself is (SLSA v1.2 specification).

The Build Track runs four levels. **Level 0** is no guarantee at all, the default for an unmanaged local build. **Level 1** requires a consistent build process and distributed provenance, but places no authenticity or accuracy requirements on that provenance. **Level 2** adds authentic provenance generated by a hosted build platform. **Level 3** requires unforgeable provenance and isolated

builds, materially raising the cost of interference by a tenant-controlled build step (SLSA v1.2 Build requirements). These levels describe provenance integrity and build-platform properties; they do not certify that the source code is non-malicious.

SLSA's threat model names threats across source, build, distribution, and consumption; mapping the earlier sections of this chapter onto them shows where each incident type sits. Unauthorized changes and source-repository compromise cover a maintainer account takeover like `event-stream` or `chalk` / `debug`; compromised dependencies cover the transitive-pull problem from Section 2.3; build-process compromise and modified-package upload cover the tj-actions pattern; registry and package compromise cover the distribution risks from Section 2.5 (SLSA v1.2 threats and mitigations).

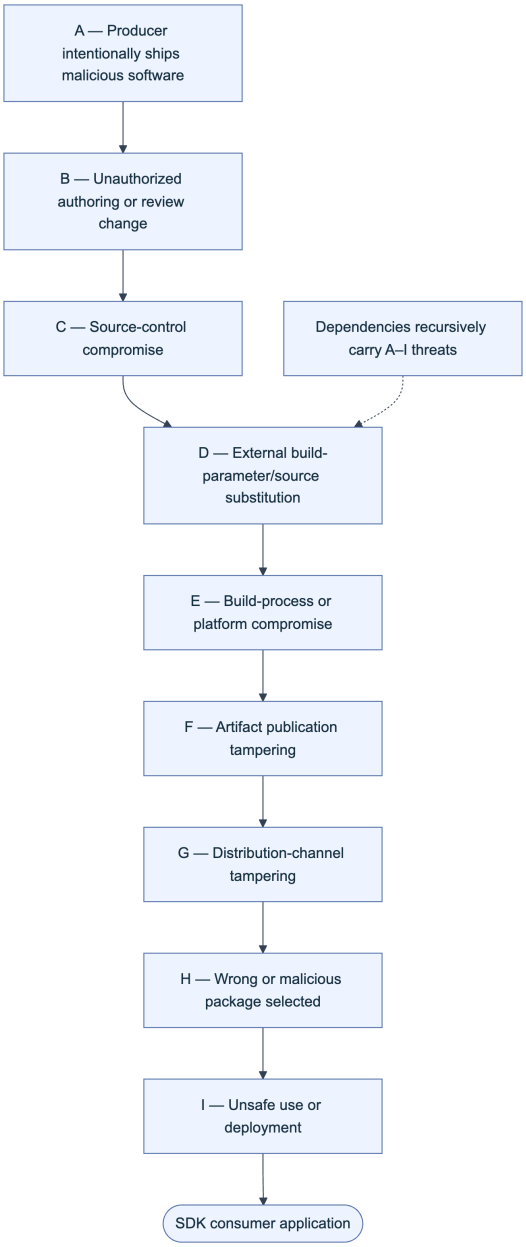

Figure 3. SLSA v1.2's A–I threat locations across producer intent, source, build, publication, distribution, selection, and use. Dependency chains recursively contain the same threat set. SLSA does not claim to mitigate every row—for example, it does not make an intentionally malicious producer trustworthy. Source model: SLSA v1.2 supply-chain threats.

For an SDK consumer, the practical output of this model is a per-dependency decision, not a single project-wide policy:

SDK Criticality Tier	Example	Minimum Build Level to Require	Verification Step
Signs or holds keys	Wallet connector SDKs	SLSA Build L3 plus signed provenance	Verify provenance attestation before every upgrade (Chapter 6)

SDK Criticality Tier	Example	Minimum Build Level to Require	Verification Step
Constructs transactions	Contract-interaction SDKs	SLSA Build L2 minimum	Diff released source against the published package before pinning
UI/front-end only	Component and hook libraries	SLSA Build L1, acceptable with hash pinning	Hash-pin in the lockfile, monitor for unexpected updates
Build-time/dev-only	Linters, test tooling	SLSA Build L1	Standard dependency review, isolate from production build secrets

Key controls for Part 1

- Classify every SDK in an application by type (wallet, contract-interaction, front-end, mobile) before assessing it; each carries a different blast radius.
- Draw the four trust boundaries (import, update, signing, network) explicitly for any SDK a team relies on, rather than assuming the boundary is safe by default.
- Track the actor behind a given risk, malicious publisher, compromised maintainer, nation-state, or internal error, because the control that stops one rarely stops another.
- Run the STRIDE scenarios in Section 2.4 against any SDK handling signing, funds, or sensitive data before adoption.
- Require a minimum SLSA Build Track level proportional to what the SDK touches, and verify provenance attestations rather than trusting a registry listing alone.

Part II: Testing Methodology

This part is the working reference for testing a software development kit (SDK) before it ships inside your application's trust boundary. It moves from a general testing framework through supply chain verification, API and interface hardening, and into the three SDK families a Web3 team is most likely to integrate: wallet and key-handling SDKs, contract interaction SDKs, and front-end or mobile SDKs. Each chapter ends with the checklist a reviewer runs against a candidate package.

3. Testing Framework Overview

An SDK is code you did not write, running with the access your application grants it. No single technique covers that risk on its own, so this chapter lays out four complementary testing lanes and where each one catches what the others miss.

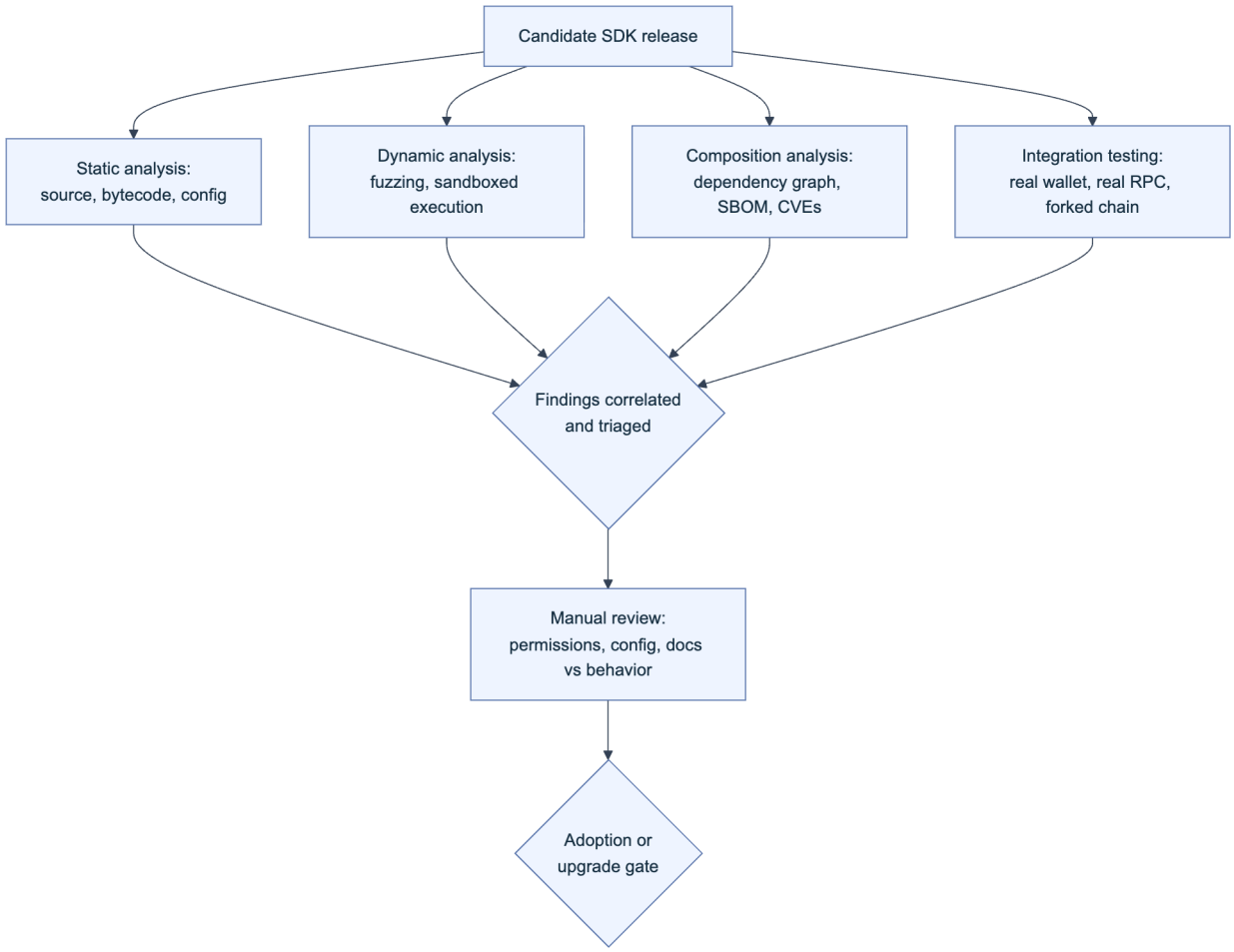

Figure 1. The four testing lanes converge on a manual review gate before an SDK version is adopted or upgraded. No lane substitutes for another: static analysis cannot see runtime network calls, and dynamic testing cannot see a typosquatted transitive dependency that never executes during the test run.

3.1 Static and Dynamic Testing

Static analysis examines the SDK's source or compiled artifact without executing it. For JavaScript and TypeScript SDKs, this means running a linter and a security-focused static analyzer such as Semgrep or ESLint security plugins against the package's shipped source (not just its published `d.ts` types), looking for dangerous sinks: `eval`, dynamic `require`, unchecked `child_process` calls, and network calls to hosts not documented in the SDK's own README. For compiled or bundled artifacts, decompile or at minimum diff the published bundle against the tagged source release; a mismatch between the two is itself a finding, since it means the artifact you would ship is not the artifact anyone reviewed.

Dynamic analysis executes the SDK under controlled conditions and observes what it actually does: which hosts it contacts, what it writes to disk or browser storage, and how it behaves under malformed input. Run the SDK's own test suite under a network-monitoring proxy (mitmproxy or a similar intercepting proxy) and diff the observed egress against the documented API surface; an unexpected call to a telemetry or analytics endpoint is a common and easy first finding. Where

the SDK exposes parsing or decoding functions (ABI decoders, QR-code parsers, deep-link handlers), fuzz them with malformed and boundary-value input rather than only the examples in the SDK's own documentation.

Technique	Catches	Misses	Representative tooling
Static source analysis	Dangerous sinks, hardcoded secrets, insecure defaults	Runtime-only behavior, obfuscated or minified logic	Semgrep, CodeQL, ESLint security plugins
Static bytecode/binary diff	Published-artifact-vs-source tampering	Behavior that only triggers under live network conditions	`diff` , reproducible-build tooling (Section 4.6)
Dynamic network monitoring	Undocumented egress, telemetry, exfiltration	Logic-only bugs with no network side effect	mitmproxy, Burp Suite, OS-level pcap
Fuzzing of parsers/decoders	Crashes, malformed-input handling, injection via decoded values	Business-logic flaws that require valid, well-formed input	Jazzer.js, AFL-style fuzzers, property-based testing (fast-check)

3.2 Composition and Dependency Analysis

Composition analysis asks a different question than static or dynamic testing: not "is this code safe" but "what is this package actually made of." An SDK's own source can be clean while a transitive dependency three levels down carries a known vulnerability or an active backdoor, which is precisely the shape of the `@solana/web3.js` compromise of December 2024, where a compromised maintainer account on npm pushed malicious versions 1.95.6 and 1.95.7 that exfiltrated private key material to a hardcoded wallet address, exposed to roughly 350,000 weekly downloads before removal (Socket.dev).

Generate a full dependency inventory before you generate the SBOM the rest of this chapter builds on:

```
# Node.js/npm: full transitive tree, including dev dependencies
npm ls --all --json > dependency-tree.json

# Rust/Cargo: transitive tree with duplicate-version detection
cargo tree --duplicates

# Python: installed package graph
pip install pipdeptree && pipdeptree --json-tree > dependency-tree.json

# Go: full module graph
go mod graph > dependency-graph.txt
```

Feed the tree into a composition scanner (Section 4.5) and flag three patterns specifically: packages with a single maintainer and no recent commit activity, packages whose version jumped by more than a minor release with no corresponding changelog entry, and any dependency re-

solved from a registry other than the one your lockfile expects. Composition analysis is only as good as the depth it reaches; a scanner that stops at direct dependencies would have missed the compromised package sitting two hops into a wallet-adapter's own dependency tree.

3.3 Integration and Behavioral Testing

Static and composition analysis tell you what the SDK could do; integration testing tells you what it does when wired into something that resembles production. Stand the SDK up against a forked mainnet or testnet (Foundry's `anvil --fork-url`, Hardhat's `hardhat_reset` with a `forking` block, or a Solana `test-validator` cloned from mainnet state) and drive it through its full lifecycle: initialization, a signing operation, a contract call, and teardown. Behavioral testing extends this by asserting on side effects the SDK does not document: does closing the SDK instance clear key material from memory, does a failed transaction leave a stale nonce lock, does a network timeout retry silently or surface to the caller.

Pair this with a mocked-provider harness so you can test failure paths that a live fork cannot easily reproduce: a wallet provider that rejects every signature request, an RPC endpoint that returns malformed JSON, a hardware wallet transport that disconnects mid-transaction. An SDK that throws an unhandled promise rejection or, worse, silently swallows the error and returns a default value under any of these conditions has failed integration testing even if every unit test in its own suite passes, because the failure mode is exactly the one an attacker or a flaky network will trigger in production.

3.4 Manual Review and Configuration Testing

Automated tooling finds what it is programmed to look for; manual review finds what the tooling was never told to expect. A reviewer reads the SDK's default configuration and asks whether each default is secure by default: does the SDK default to verifying TLS certificates, does it default to the safest signing mode available (typed data over raw hex), does it default to a minimal permission or scope request. Cross-check the SDK's documented behavior against its actual behavior line by line for the security-relevant claims specifically (key storage location, network endpoints contacted, data retained between sessions); a documentation gap here is itself a finding, because a security team cannot review a control the vendor never disclosed.

Configuration testing walks every exposed option the SDK accepts and asks what happens at each extreme: an empty API key, a malformed chain ID, a negative gas limit, a callback URL pointing at `localhost`. Record findings against this checklist:

- [ ] Every documented default is the secure option, not the permissive one
- [ ] Undocumented configuration options do not exist, or are flagged as internal/unstable
- [ ] TLS verification cannot be silently disabled by a single boolean without a corresponding warning log
- [ ] Debug or verbose logging modes do not print secrets, seed phrases, or full transaction payloads to stdout
- [ ] The SDK fails closed (rejects the operation) rather than fails open (proceeds with defaults) on invalid configuration

---

4. Supply Chain and Dependency Testing

The previous chapter's composition analysis identifies what an SDK depends on; this chapter defines the evidence formats and verification chain that let you trust what you found, rather than merely list it. Supply chain risk in the OWASP Top 10:2025 is significant enough to warrant its own top-level category, A03 Software Supply Chain Failures, ahead of injection and authentication failures in the current ranking (OWASP Top 10:2025).

4.1 Identifying Direct and Transitive Dependencies

Direct dependencies are the packages an SDK's manifest names explicitly; transitive dependencies are everything those packages pull in, recursively, and this second set is both far larger and far less scrutinized. A typical wallet-adapter SDK might declare a dozen direct dependencies and resolve to several hundred transitive ones, and the Ledger Connect Kit compromise of December 2023 demonstrated that the depth of the tree does not correlate with the depth of the damage: a single compromised package, however deep, propagated to every dApp that imported it without pinning. A compromise three hops deep (an image parser pulled in by a QR decoder pulled in by a wallet adapter) has the same blast radius as a compromise in a direct dependency, because every consumer of the top-level SDK inherits it regardless of depth.

Testing at this level means generating the tree (Section 3.2 commands), then explicitly walking every leaf node that has write access to disk, network access, or that runs during install (`postinstall` scripts are a disproportionately common attack vector and should be enumerated separately: `npm ls --all --json | jq` filtered for packages with lifecycle scripts, or `npm config set ignore-scripts true` as a default posture with explicit allowlisting).

4.2 SBOM: CycloneDX 1.6, SPDX 3.0, VEX (Vulnerability Exploitability)

A software bill of materials (SBOM) is a machine-readable inventory of everything a piece of software is built from: every direct and transitive dependency, its version, its license, and increasingly its build provenance. Two formats dominate current practice. CycloneDX, maintained under OWASP, reached version 1.6 in April 2024 and was standardized as ECMA-424 1st Edition, extending coverage beyond software components to hardware, services, cryptographic assets, and AI models, with build formulation and assembly-completeness fields relevant to verifying an SDK's own build. SPDX 3.0, an ISO/IEC 5962 lineage standard released in April 2024, restructured the format around a Resource Description Framework (RDF) model with eight independent profiles (Core, Software, Security, Build, AI, Dataset, Licensing, and Lite), letting a consumer request only the Security profile of an SBOM without parsing the rest.

Neither format alone tells you whether a listed vulnerability actually affects the SDK as integrated. That is the job of VEX (Vulnerability Exploitability eXchange), a companion document format that annotates each SBOM entry's known vulnerabilities with an exploitability status,

defined by the CISA VEX working group and implemented in profiles such as OpenVEX: `affected`, `not_affected`, `fixed`, or `under_investigation`. Without VEX, a security scanner flags every CVE in the dependency tree regardless of whether the vulnerable code path is reachable, producing the false-positive volume that makes teams stop reading scanner output altogether.

```
{
  "vulnerability": "CVE-2024-XXXXX",
  "product": "pkg:npm/example-wallet-sdk@2.4.1",
  "status": "not_affected",
  "justification": "vulnerable_code_not_in_execute_path",
  "impact_statement": "Vulnerable function is only reachable via a legacy XML import
        path removed in this build configuration."
}
```

Require an SBOM in CycloneDX 1.6 or SPDX 3.0 as a condition of SDK adoption (Section 10 of Part III), and require VEX statements for any Critical or High severity vulnerability the SBOM's own composition scan surfaces, rather than accepting a bare "we're aware of it."

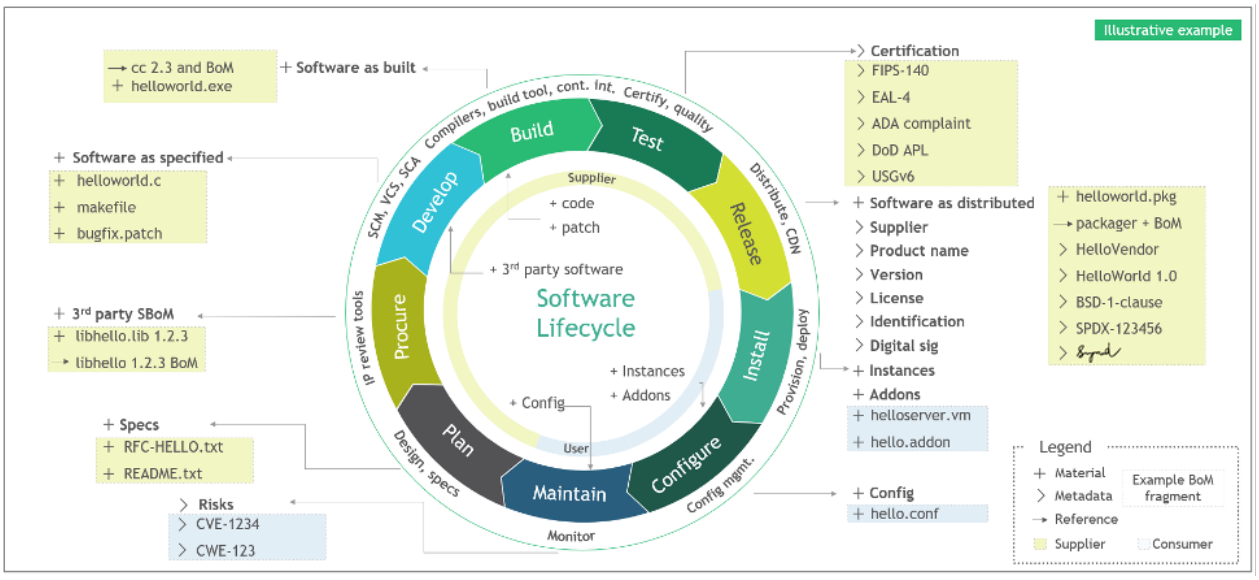

Figure. NIST's SBOM assembly-line model shows why a single package list at release time is incomplete: materials, metadata, and references accumulate throughout procurement, development, build, testing, release, installation, and configuration. An SDK assessment should preserve those relationships, not flatten them into names and versions. Source: NIST, Software Security in Supply Chains, Appendix F Figure 2, public domain (U.S. government work).

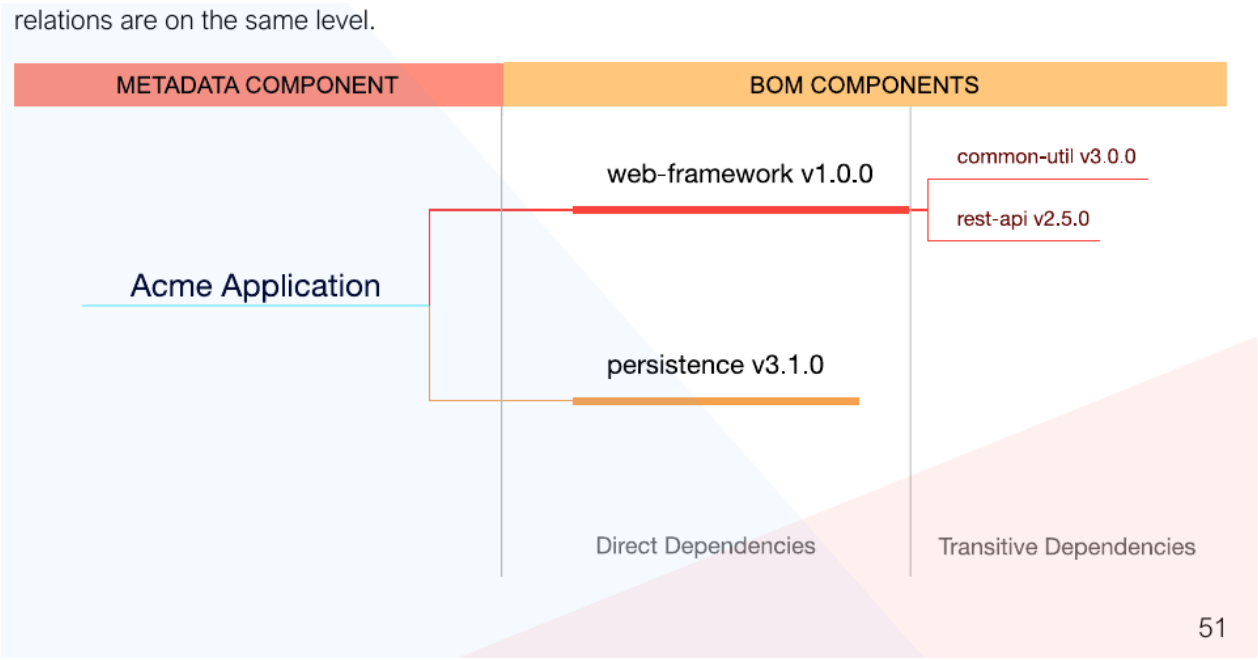

Figure. CycloneDX preserves dependency relationships instead of treating an SBOM as a flat package inventory. That graph is essential when a vulnerable component is present but only reachable through a particular SDK or build path. Source: OWASP CycloneDX, Authoritative Guide to SBOM, reproduced with attribution to the OWASP CycloneDX project.

4.3 SLSA v1.2: Build and Source Tracks, Provenance, and Verification

Supply-chain Levels for Software Artifacts (SLSA, pronounced "salsa") defines graduated requirements for establishing verifiable properties of software source and builds. SLSA v1.2 is the current approved specification and supersedes the retired v1.0 material. It has two independent tracks relevant to SDK assurance: the **Build Track** (L0–L3), which measures provenance integrity and build-platform isolation, and the **Source Track** (L0–L4), which measures version control, history and source provenance, continuous technical controls, and two-party review. An SDK evaluation should record both tracks rather than using "SLSA level" as an ambiguous single score.

Provenance is metadata **about** the artifact: a structured statement binding an output digest to source, builder, and build parameters. It is not the package itself and does not prove that the source is benign. Verification is the consumer-side act of authenticating that statement, confirming the subject digest matches the artifact about to be installed, checking builder/source identities and policy, and rejecting an absent, malformed, unexpected, or unverifiable attestation rather than trusting its claims at face value.

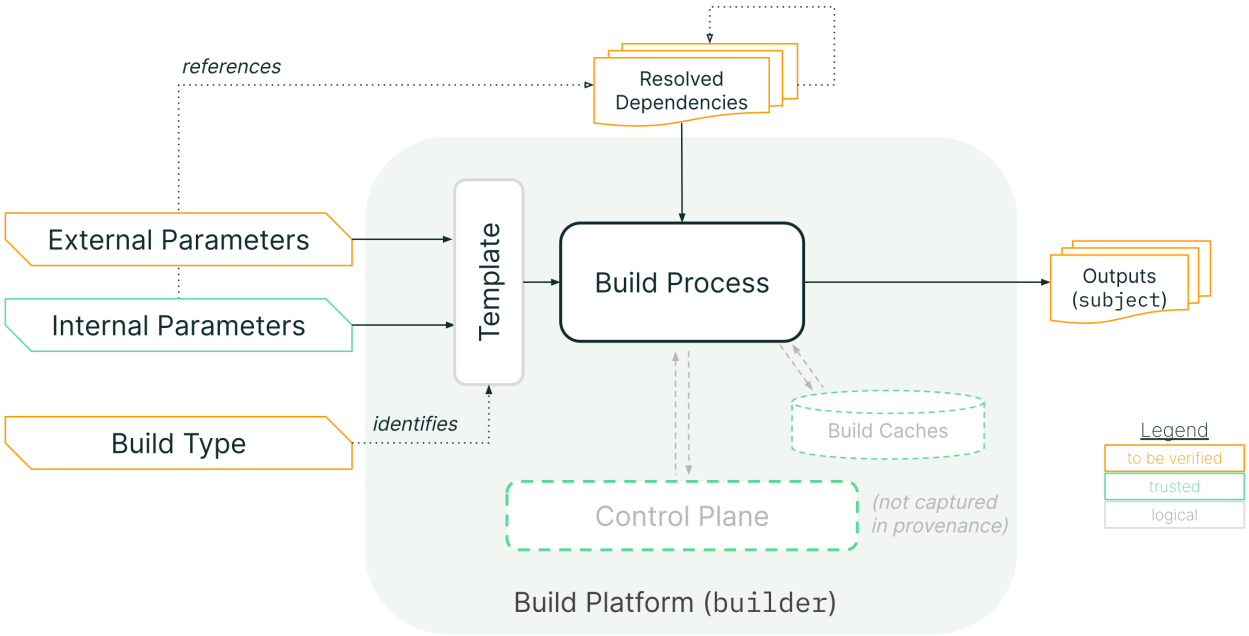

Figure. The SLSA provenance model separates caller-controlled external parameters from builder-resolved dependencies and binds both to the output artifact. Reviewers should verify the provenance predicate and builder identity, not merely check that an attestation file exists. Source: SLSA Build Provenance, Community Specification License 1.0.

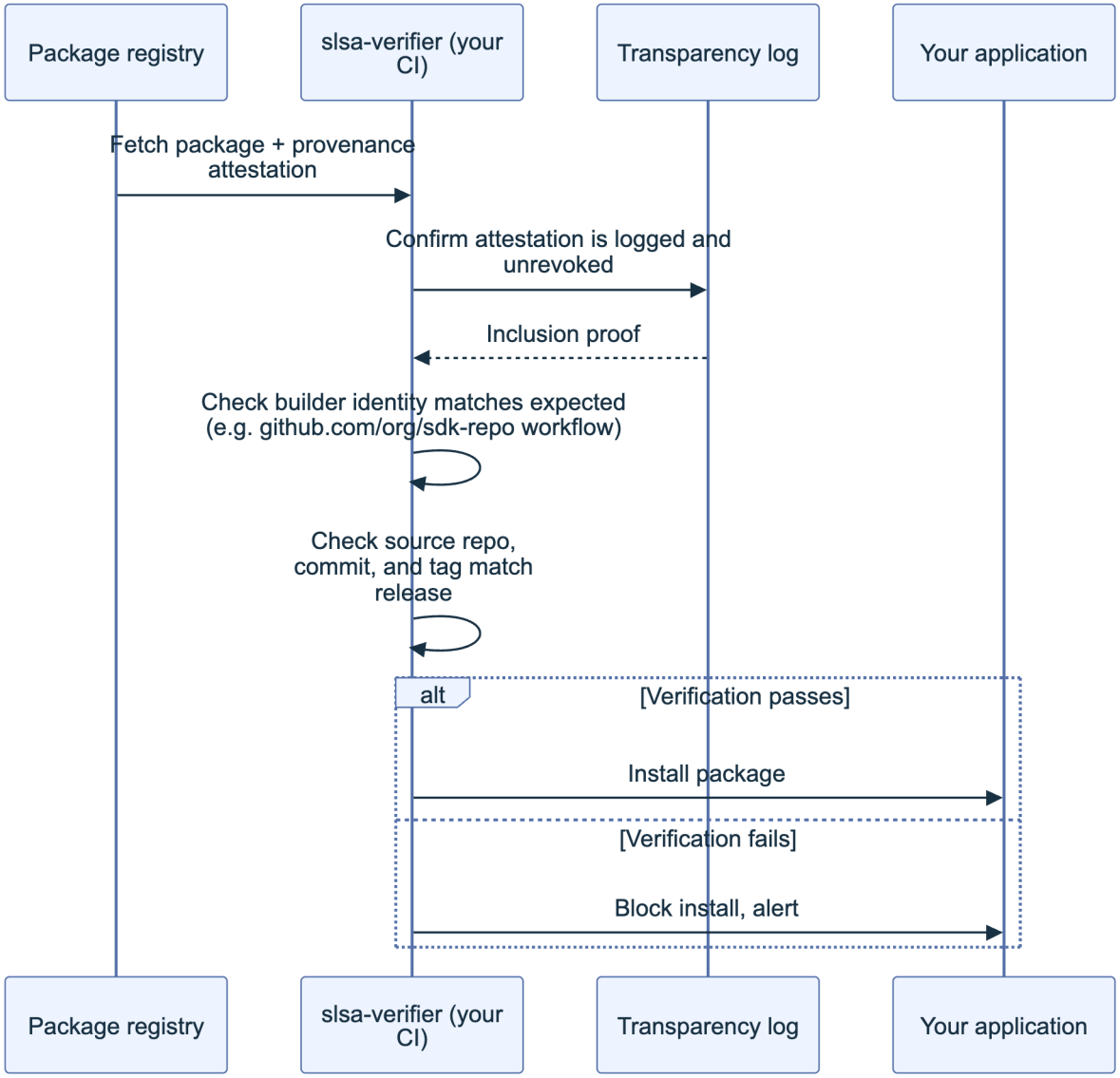

Figure 2. Consumer-side SLSA verification gate. The check runs in your own CI/CD before the SDK is installed, not once at initial adoption, so a later malicious release with unmatched or absent provenance is caught on every build.

Bake this verification step into the CI/CD pipeline that pulls in the SDK (Part III, Chapter 9), not just into a one-time vendor assessment: a package that shipped SLSA-compliant provenance at version 2.0.0 can still ship a compromised, unattested version 2.0.1 if nothing checks every install.

4.3.1 Build L0–L3 and Source L0–L4 in v1.2

The Build Track retains L0 through L3 in v1.2. The Source Track added in v1.2 is separately assessed from L0 through L4; its L4 requires two trusted persons to agree to changes to protected branches. A producer can therefore be Build L3 but Source L1, or Source L4 but Build L0. Neither track should be inferred from repository badges or CI-provider marketing—verify the attestations and the platform assessment against policy.

Level	Requirement	What it proves	Typical SDK posture
L0	None	Nothing; development or test builds only	A maintainer's laptop `npm publish` with no CI involvement
L1	Consistent build process; provenance exists and is distributed	The output is digest-bound to descriptive provenance, but the specification imposes no authenticity or accuracy requirement at this level	CI emits provenance that the consumer treats as informational
L2	All L1, plus authentic provenance generated by a hosted build platform	A verifier can authenticate the attestation and identify the platform/control plane that issued it	An assessed hosted builder with policy-verified identity and signed provenance
L3	All L2, plus unforgeable provenance and isolated builds	Tenant-controlled build steps cannot forge provenance or influence another build under the assessed platform model	A documented SLSA Build L3 platform and generator, verified against its assessment

Treat L2 as the practical minimum bar for a wallet or contract-interaction SDK your team ships to end users, and treat the absence of any provenance at all (effectively L0) as a finding to raise during SDK evaluation (Part III, Chapter 10), not a detail to note in passing.

Source level	Required property	SDK-consumer evidence
L0	No Source Track claim	No trustworthy Source VSA
L1	Source is version controlled	Source VSA identifies an immutable revision
L2	Continuous history and source provenance	Source provenance records who/what changed protected references and preserves ancestry
L3	Continuous enforcement of declared technical controls	Attestations show branch/tag protections and organization-defined controls remained in force
L4	Two-party review for protected-branch changes	Evidence binds the final revision to approval by at least two trusted persons

For high-impact SDKs, pair Build L2 or L3 with Source L3 or L4. Build provenance cannot compensate for unilateral, malicious source changes, while reviewed source cannot compensate for a compromised or non-verifiable build path.

4.4 Signed Provenance and Sigstore (Fulcio, Rekor, Cosign)

Provenance is only trustworthy if it is signed by an identity you can verify, and signed by a key that was not itself sitting in a CI secret waiting to be exfiltrated. Sigstore solves this with keyless signing: a CI job authenticates to an OpenID Connect (OIDC) identity provider (proving it is, for example, the specific GitHub Actions workflow at a specific commit), Fulcio issues a short-lived certificate binding a freshly generated, ephemeral key to that identity, Cosign uses the ephemeral key to sign the artifact and then discards it, and Rekor records the signing event in a public, append-only transparency log. No long-lived private key exists for an attacker to steal, and every signature is independently auditable against the log.

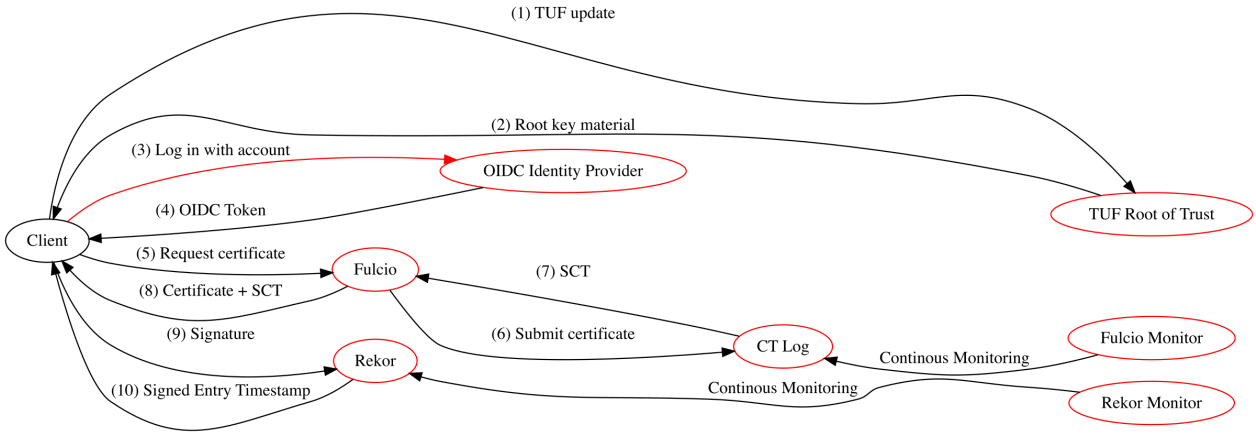

Figure. Sigstore's signing trust model makes the independent verification points explicit: workload identity, short-lived certificate issuance, transparency-log inclusion, registry integrity, and verifier policy. A pipeline that validates only the signature but not the expected OIDC identity leaves the central authorization question unanswered. Source: Sigstore Threat Model, official Sigstore project documentation, Apache-2.0.

For an SDK you consume, verification is a single command run in CI before install, checking both the signature and the identity that produced it:

```
# Verify an SDK release was signed by the expected CI workflow identity,
# and that the signing event is present in the public transparency log
cosign verify \
  --certificate-identity-regexp "^https://github.com/example-org/wallet-sdk/.github/
        workflows/release.yml@refs/tags/v.*$" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com" \
  ghcr.io/example-org/wallet-sdk:2.4.1
```

A verification failure here (wrong identity, missing signature, or an artifact absent from Rekor) should hard-fail the build, the same posture as an SLSA provenance mismatch. The two checks are complementary: SLSA provenance describes what was built and how; Sigstore proves who attested to it and that the attestation has not been silently altered since.

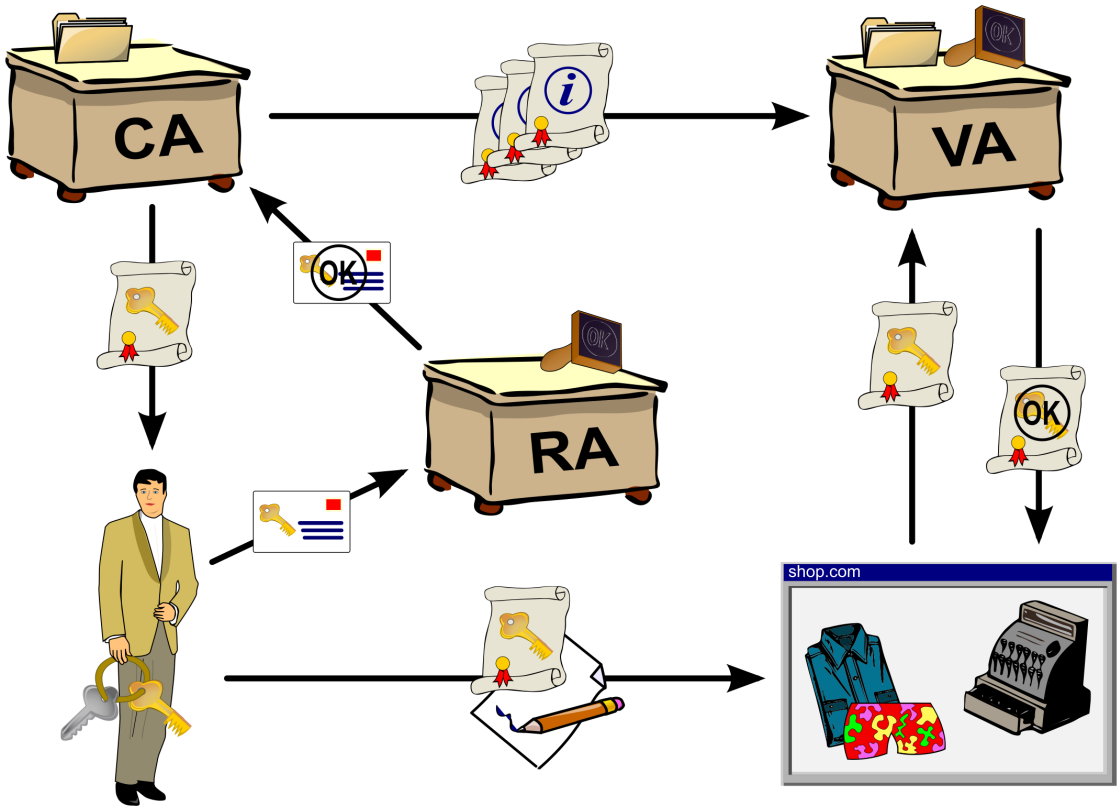

Figure. The traditional certificate-issuance model a public key infrastructure (PKI) formalizes: an identity is vetted, then a certification authority issues a certificate binding that identity to a key. Sigstore's Fulcio automates this same role for CI workflows, issuing a short-lived certificate bound to an OIDC identity instead of a long-term key a maintainer must protect. Source: Wikimedia Commons, CC BY-SA 3.0.

4.5 Vulnerability Databases and Continuous SCA

Software composition analysis (SCA) is only as current as the vulnerability data it queries against, so treat the database choice as part of the control, not an implementation detail. OSV.dev, maintained by Google's Open Source Security team, aggregates ecosystem-specific advisories (npm, PyPI, crates.io, Go, and more) in a schema designed for automated tooling rather than human reading. The GitHub Security Advisory (GHSA) database feeds directly into `npm audit` and Dependabot for packages hosted on GitHub-adjacent registries. The National Vulnerability Database (NVD), maintained by NIST, provides the canonical Common Vulnerabilities and Exposures (CVE) records that both feed and are fed by the others.

Run composition analysis as a continuous CI gate, not a point-in-time audit, since a dependency that was clean at integration time can have a CVE published against it the following week:

```
# OSV-Scanner against a lockfile, machine-readable output for CI gating
osv-scanner --lockfile=package-lock.json --format=json > osv-results.json

# Fail the build on any Critical or High finding without a VEX "not_affected" statement
jq '[.results[].packages[].vulnerabilities[]? |
     select(.severity[]?.score >= 7.0)] | length > 0' osv-results.json
```

Schedule a second, independent run on a cron trigger (daily is typical) against the dependency tree of every SDK already in production, not only at build time, so that a newly disclosed vulnerability in a package you integrated eighteen months ago surfaces without waiting for your next dependency bump.

4.6 Reproducible Builds and Integrity

A reproducible build is one where building the same source, with the same declared toolchain and inputs, produces byte-for-byte identical output regardless of who runs the build or when (Reproducible Builds project). This closes a gap that signing alone leaves open: signing proves an identity attested to an artifact, but says nothing about whether that artifact actually corresponds to the public source it claims to be built from. A malicious insider with legitimate signing access, or a compromised build server, can sign a tampered artifact and pass every provenance and Sigstore check described above.

Reproducibility turns that gap into something independently checkable: any third party can pull the tagged source, run the documented build, and compare the resulting hash against the one the vendor published and signed.

```
# Rebuild an SDK release from tagged source and diff against the published artifact
git clone --branch v2.4.1 https://github.com/example-org/wallet-sdk
cd wallet-sdk && npm ci && npm run build

sha256sum dist/wallet-sdk.min.js
# Compare against the hash published in the release notes / provenance attestation
```

Web3 tooling already has a working precedent for this discipline in contract verification: Sourcify and Etherscan's bytecode-matching verification both depend on a deterministic Solidity compilation producing bytecode that matches what was deployed on chain, and the same rebuild-and-diff logic applies directly to an SDK's JavaScript bundle or native binary. Where an SDK vendor cannot reproduce their own published build, treat that as a supply chain finding: it means neither you nor they can prove the shipped artifact matches the reviewed source.

4.7 Version Pinning, Private Registry, and Sourcing Policies

Every control in this chapter assumes you know exactly which version of an SDK is running, which means floating version ranges (`^2.4.0`, `latest`, or an unpinned Git branch reference) undermine all of them at once: a caret range can silently resolve to a compromised patch release the moment it publishes, before your next scheduled SBOM or SLSA check even runs. Pin exact versions in the manifest and commit the lockfile (`package-lock.json`, `Cargo.lock`, `poetry.lock`) so that every install, in every environment, resolves identically:

```
{
  "dependencies": {
    "wallet-sdk": "2.4.1"
  }
}
```

For SDKs that handle keys or construct transactions, consider mirroring the exact pinned version, plus its verified SBOM and provenance, into a private registry (a self-hosted Verdaccio instance, or a scoped Artifactory/Nexus repository) rather than resolving directly from the public registry on every build. This does two things: it makes a later upstream removal or a targeted "some versions only" compromise (where an attacker serves malicious bytes only to specific IP ranges or on a specific time window, as seen in the Polyfill.io case referenced in Handbook 01) unable to reach your build pipeline at all, and it gives you a single point to enforce the version-pinning and provenance-verification policy rather than relying on every engineer's local environment to do so correctly.

A sourcing policy formalizes this as a written gate: no SDK is added to the dependency tree without a documented owner, a minimum SLSA level, and a named individual responsible for tracking its advisories, mirroring the vendor evaluation criteria detailed in Part III, Chapter 10.

---

5. API and Interface Security

Once an SDK's supply chain is verified, its runtime interface becomes the next surface: every parameter it accepts, every credential it handles, and every error it surfaces is a place where the SDK's own implementation, not a compromised dependency, can be the vulnerability.

5.1 Input Validation and Sanitization

An SDK sits between untrusted external input (user-supplied addresses, amounts, RPC responses, deep-link parameters) and your application's trust boundary, and it is not automatically safe to assume the SDK validates that input before acting on it. Test every public method with boundary and malformed values specifically: a negative or fractional token amount where only positive integers are valid, an address string with mixed case that fails EIP-55 checksum validation, a chain ID that does not match any network the SDK claims to support, and an oversized payload (a multi-megabyte string passed where a 42-character address is expected).

```
// Minimal harness for boundary-value input testing against an SDK method
const boundaryInputs = [
  "", null, undefined, "0x", "-1", "1.5",
  "0".repeat(10000),                    // oversized
  "0xZZ...",                            // invalid hex
  "0x" + "f".repeat(39),                // one char short
];
for (const input of boundaryInputs) {
  try {
    const result = await sdk.transferTo(input, 1n);
    console.log(`ACCEPTED (check if this should have been rejected): ${input}`);
  } catch (e) {
    console.log(`Rejected: ${input} -> ${e.message}`);
  }
}
```

An SDK that accepts any of these without a clear, typed rejection is pushing the validation burden onto your application code silently, and if your application also assumes the SDK validated it, the gap is invisible until an attacker finds it.

5.2 Authentication and Secret Handling in SDK

SDKs that call an API on the developer's behalf (price oracles, indexing services, relayer networks) need a credential, and how that credential is requested, stored, and scoped is a direct security property of the SDK. Review whether the SDK's initialization API accepts a scoped, revocable API key or demands a broad credential; whether it will accept the key as a plain constructor argument that ends up in client-side bundles (a critical failure for any front-end-facing SDK, since a key shipped to the browser is public) versus requiring it be injected server-side; and whether the SDK ever logs the credential, even at debug verbosity.

Anti-pattern	Why it's dangerous	Secure alternative
API key hardcoded in SDK constructor call within front-end bundle	Key is extractable from any user's browser dev tools	Server-side proxy; browser SDK receives a short-lived, scoped session token
SDK logs full request/response including auth headers at `debug` level	Logs shipped to third-party observability tools leak credentials	Redact auth headers unconditionally, even in debug mode
Single API key with full account privileges, no scoping	Compromise of the key compromises everything the account can do	Per-integration, least-privilege keys with read/write scope split
No key rotation mechanism exposed by the SDK	Rotation requires a full redeploy, so it happens rarely if ever	SDK supports hot-swapping credentials without reinitialization

Confirm the SDK supports credential rotation without requiring a full application restart; an SDK that caches a credential for the life of the process makes timely revocation operationally painful, which in practice means it happens late or not at all.

5.3 Error Handling and Information Disclosure

Errors are a rich, frequently overlooked information-disclosure channel. Trigger every failure path you can reach (network timeout, malformed RPC response, insufficient balance, contract revert, rate limiting) and inspect exactly what the SDK's error object contains: a well-behaved SDK returns a typed, minimal error; a poorly behaved one returns the full underlying HTTP request including any authorization header, the raw RPC endpoint URL (which may itself be a paid, rate-limited endpoint you did not intend to expose), or a stack trace referencing internal file paths and package versions useful for an attacker fingerprinting your stack.

```
// Bad: SDK error leaks the underlying RPC endpoint and full request context
try {
  await sdk.sendTransaction(tx);
} catch (e) {
  console.error(e);
  // e.message: "Request to https://mainnet.infura.io/v3/9f8a...c21 failed: 429"
}

// What a well-scoped SDK error should expose
try {
  await sdk.sendTransaction(tx);
} catch (e) {
  console.error(e.code, e.message);
  // e.code: "RATE_LIMITED", e.message: "Transaction submission rate limited, retry
       after 2s"
}
```

Where the SDK's error handling is too verbose to fix quickly, wrap every call at your own application boundary and re-throw a sanitized error, but log the finding against the SDK regardless: an information-disclosure gap in a widely used SDK affects every application that has not built the same wrapper.

5.4 Versioning and Breaking Changes

An SDK's versioning discipline is a security control, not just a developer-experience concern, because a team that cannot tell a security patch from a breaking change from a routine feature bump either upgrades too slowly (staying exposed to a known, published vulnerability) or upgrades blindly (absorbing an unreviewed behavioral change into a production signing path). Confirm the SDK follows semantic versioning strictly: patch releases contain only backward-compatible fixes, minor releases add functionality without breaking existing calls, and major releases are the only place breaking changes are permitted, each with a changelog entry explaining the change and, where security-relevant, a corresponding CVE or advisory.

Build a versioning review into your upgrade process:

- ☐ Read the changelog for every version between your current pin and the target, not only the target version's own notes
- ☐ Treat any patch release with a security advisory as a required, expedited upgrade, decoupled from routine feature upgrades
- ☐ Re-run the full test suite in Chapter 3 against every major version bump before it lands in production, since a major release is exactly where an SDK's default security posture is most likely to change
- ☐ Confirm deprecated APIs the SDK still exposes carry an explicit sunset date, so your integration does not silently depend on code the vendor has stopped maintaining or testing

6. Wallet and Key-Handling SDKs

Wallet and key-handling SDKs carry the highest blast radius of any SDK category in this handbook: a flaw here does not leak data, it moves funds. This chapter treats key generation, hardware wallet interaction, and phishing resistance as the three properties a wallet SDK must get right, in that order of difficulty to verify.

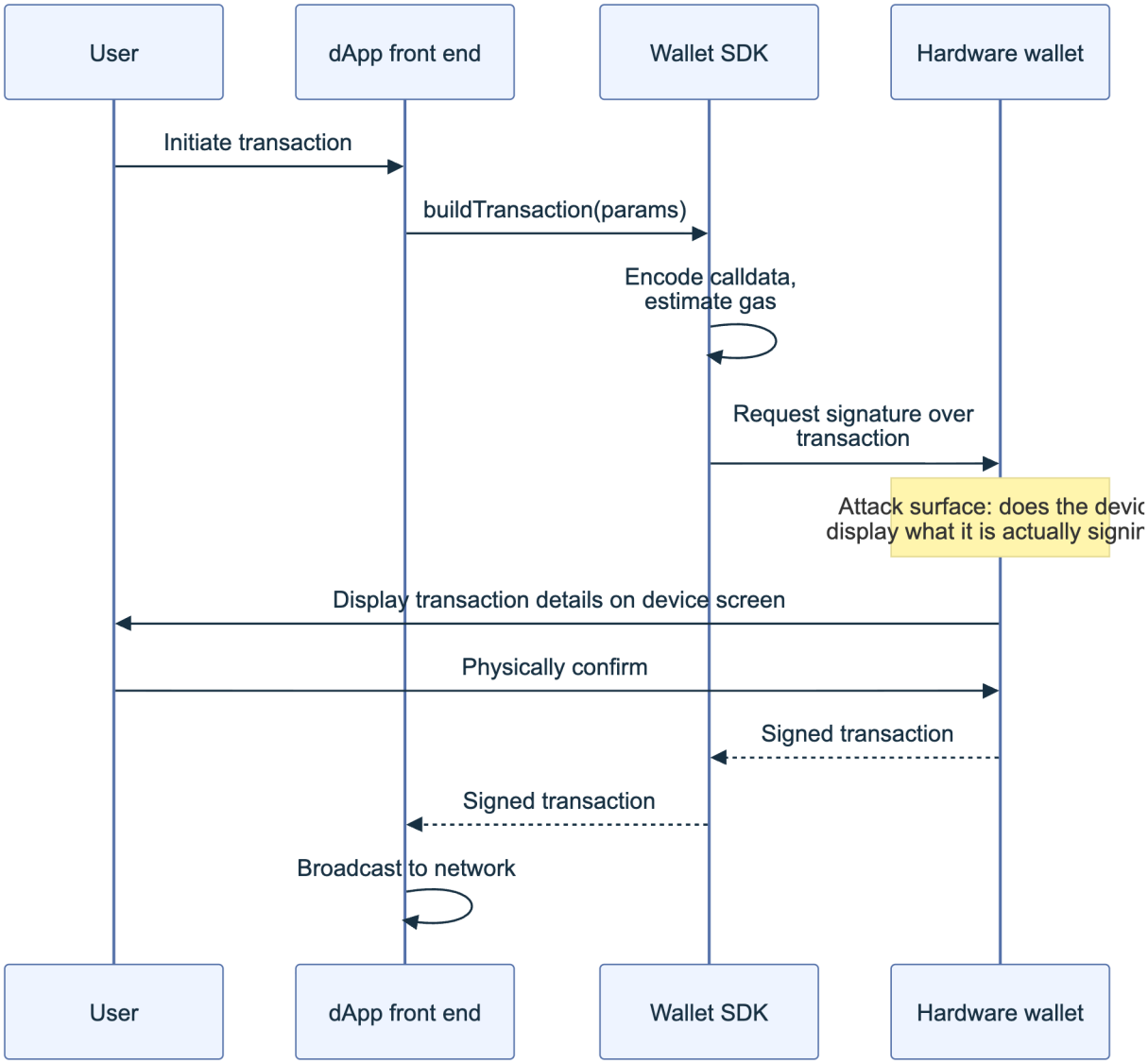

Figure 3. The wallet SDK signing path. The single highest-value verification point is whether the data displayed on the hardware wallet's own screen matches the data actually being signed; a compromised SDK or a compromised front end can construct one transaction while displaying another if this link is not independently enforced.

6.1 Key Generation, Storage, and Signing

Test a wallet SDK's key generation against the strength of its underlying randomness source before anything else, because a flaw here is silent and total: it does not cause a crash or a visible error, it simply produces predictable keys that an attacker can reproduce offline. The Profanity vanity address generator is the canonical cautionary case: it seeded its pseudo-random number generator with a 32-bit unsigned integer, a space of roughly 4.3 billion possible seeds, small enough that a modest GPU cluster could brute-force the private key behind any address the tool generated. Market maker Wintermute had used a Profanity-generated vanity address for an operational wallet and lost approximately $160 million when attackers reconstructed its private key on September 20, 2022 (The Hacker News).

Verify that a candidate SDK sources randomness from the platform's cryptographically secure generator (`crypto.getRandomValues` in browsers, `crypto/rand` in Go, `os.urandom` / `secrets` in Python) and never from a seeded, deterministic, or user-suppliable source unless explicitly and clearly labeled as a test-only mode. For storage, confirm private key material is held in the platform's dedicated secure storage (OS keychain, Trusted Execution Environment, hardware secure element) rather than plain memory, browser `localStorage`, or an application-level variable that a debugger or a separate malicious script on the same page could read. Signing itself should be tested for confirmation that the SDK never has access to a raw, unencrypted private key when a hardware wallet or secure enclave is in use; the SDK's role in that path should be limited to constructing and forwarding the transaction, never holding the key.

6.2 Interaction with Hardware and External Wallets

Hardware wallets and external wallet applications (MetaMask, Phantom, and similar browser-extension or mobile wallets) are separate trust domains from the SDK itself, communicating over well-defined transports: WebUSB and WebHID for direct browser-to-device communication, or an injected provider object (`window.ethereum`, the EIP-1193 provider interface) for browser-extension wallets. Test that the SDK correctly handles a wallet that is present but locked, a wallet extension that is absent entirely (the SDK should detect this and fail informatively, not throw an unhandled exception), and a wallet that returns a user-rejected error, which should propagate as a distinct, catchable error type rather than being conflated with a network failure.

A subtler test targets provider spoofing: on a page with multiple wallet extensions installed, does the SDK correctly identify and bind to the specific provider the user selected, or does it default to whichever extension registered `window.ethereum` last, a race condition that a malicious browser extension could exploit to intercept signing requests intended for a legitimate wallet. For SDKs supporting WalletConnect-style remote pairing, verify the pairing QR code or deep link encodes a session-specific, short-lived token rather than a static or predictable identifier, and that the SDK validates the relay server's TLS certificate rather than accepting any relay that responds.

6.3 Phishing and UI Redress Risks

The most consequential wallet SDK incident to date targeted this exact category. On February 21, 2025, attackers stole more than $1.4 billion, including 401,347 ETH, from Bybit's cold wallet, in what remains one of the largest cryptocurrency thefts on record. The root cause traced back to a compromised developer machine at Safe{Wallet}, whose multisig front end Bybit used to manage the cold wallet: the attacker injected malicious JavaScript into the Safe web interface that detected when an authorized Bybit signer initiated a routine transaction, silently swapped in a malicious transaction that granted control of the wallet's logic to the attacker, and then restored the original-looking transaction data so the signer's confirmation appeared normal (NCC Group technical analysis). Every signer who approved the transaction was, in effect, blind-signing: confirming a signature over data their device or interface did not accurately represent.

This is UI redress applied to the signing path rather than to a login form, and it defeats a hardware wallet's physical confirmation step entirely if the front end feeding it is compromised, because the device faithfully displays whatever calldata it is handed. The primary mitigation is structured, human-readable signing rather than raw hex: EIP-712 typed structured data lets a wallet or hardware device render the actual fields being signed (recipient, amount, nonce) rather than an opaque byte string, and a wallet SDK should be tested for whether it defaults to EIP-712 or a chain-equivalent typed-signing standard for every transaction type it supports, falling back to raw signing only when the target contract genuinely offers no structured alternative. Test also whether the SDK surfaces a warning when a dApp requests a signature over unusually broad permissions (an unlimited token approval, a `setOwner`-style call, a proxy upgrade) rather than treating every signature request identically.

6.4 Test Cases and Checklist for Wallet SDKs

Test case	Pass condition
Key generation entropy source	Uses platform CSPRNG exclusively; no seeded or predictable fallback path exists in any code path, including error handling
Key material never leaves secure storage in plaintext	Confirmed via memory inspection or documented architecture that the SDK cannot export raw private keys when hardware/enclave signing is configured
Absent wallet handling	SDK returns a typed, catchable error; does not throw an unhandled exception or hang indefinitely
Multi-wallet provider disambiguation	SDK binds to the user-selected provider deterministically, not by registration order
Default signing mode	Defaults to EIP-712 or equivalent typed/structured signing wherever the target supports it
Broad-permission signature warning	SDK surfaces a distinct warning UI hook for unlimited approvals, ownership transfers, and upgrade calls
Session/pairing token strength	WalletConnect or equivalent pairing tokens are session-specific, short-lived, and not predictable
Signing request replay	SDK includes and validates a nonce or equivalent anti-replay field on every signing request it constructs

7. Contract Interaction SDKs

Contract interaction SDKs translate a developer's high-level call into the raw bytes a blockchain executes, and every step of that translation, encoding, gas estimation, and network selection, is a place where the SDK's correctness is directly your application's correctness.

7.1 ABI Encoding, Decoding, and Calldata

The Application Binary Interface (ABI) defines how a function call and its arguments are encoded into the raw calldata a contract receives, and an SDK's encoder is trusted to get this exactly right, because a subtly wrong encoding does not usually fail loudly; it calls a different function, or the intended function with corrupted arguments. Test the SDK's encoder against known-answer vectors first: encode a call with a fixed set of arguments and compare the output byte-for-byte against a reference implementation (`ethers.js` against `web3.py`, or either against `cast calldata` from Foundry), across the full type surface the SDK claims to support: dynamic arrays, nested structs/tuples, and multi-dimensional arrays specifically, since these are where hand-rolled encoders most often diverge from the specification.

```
# Cross-check an SDK's encoded calldata against a reference implementation
cast calldata "transfer(address,uint256)" 0x1234...abcd 1000000000000000000
# Compare byte-for-byte against the SDK's own encodeFunctionData() output
# for the same function signature and arguments
```

Function selector collisions are a related, narrower risk: the selector is only the first four bytes of the Keccak-256 hash of the function signature, so two differently named functions can (rarely, but not impossibly for an adversary willing to search) collide. Confirm the SDK resolves selectors from a full, verified ABI rather than from a bare selector-to-name lookup table it maintains independently, since a stale or incomplete lookup table can silently misroute a call. On the decoding side, fuzz the SDK's event-log and return-value decoders with truncated, oversized, and type-mismatched calldata; a decoder that throws an uncaught exception on malformed input from a malicious or non-standard contract can crash an application's transaction-monitoring pipeline.

7.2 Gas Estimation and Transaction Construction

Gas estimation sits on the path between "the SDK thinks this will succeed" and "the network actually executes it," and an SDK that gets this wrong either wastes user funds on failed transactions or, more subtly, underestimates gas in a way that succeeds during testing but fails intermittently in production under network congestion. Test estimation against contracts with state-dependent gas costs specifically (a mapping write that costs more on first use than on update, a loop whose iteration count depends on caller-supplied input) and confirm the SDK's estimate

includes a safety margin rather than the network's bare minimum estimate, since `eth_estimateGas` itself can under-report for transactions whose gas cost depends on state that changes between estimation and execution.

Verify the SDK correctly implements EIP-1559 fee fields (`maxFeePerGas`, `maxPriorityFeePerGas`) rather than only legacy `gasPrice`, and that it does not silently clamp a user-specified priority fee to zero on networks where that would cause the transaction to be deprioritized indefinitely rather than rejected outright. Nonce management deserves its own explicit test: submit two transactions from the same account in rapid succession and confirm the SDK either serializes them correctly (second transaction uses `nonce + 1`, not a stale cached nonce) or clearly documents that concurrent submission is the caller's responsibility. An SDK that silently reuses a nonce under concurrent calls produces one of the two transactions failing with a confusing "nonce too low" error that gives no indication of the actual cause.

7.3 Network and RPC Security

Every contract interaction SDK depends on a Remote Procedure Call (RPC) endpoint to reach the blockchain, and that endpoint is itself a trust boundary the SDK either protects or exposes. Test that the SDK enforces TLS on every RPC endpoint by default and either rejects or clearly warns on a plaintext HTTP endpoint, since an RPC response is the source of truth the SDK builds transactions and gas estimates from; a network-position attacker who can tamper with an unencrypted RPC response can feed the SDK a manipulated balance, nonce, or gas price. Confirm the SDK validates that the connected network's chain ID matches what the calling application expects before broadcasting, guarding against the class of attack where a malicious or misconfigured RPC endpoint added via EIP-3085 (`wallet_addEthereumChain`) silently returns responses for a different chain than the one it claims to be, tricking a user into signing a transaction they believe applies to one network when it broadcasts to another.

Where an SDK supports custom or user-suppliable RPC endpoints (common in wallet and multi-chain SDKs), test the failover and fallback behavior when a configured endpoint is unreachable, rate-limited, or returns internally inconsistent data (a block number that decreases between calls, a chain ID that changes mid-session): a well-built SDK detects this and either fails closed with a clear error or falls back to a secondary, independently verified endpoint, rather than silently continuing to build transactions against unreliable state.

7.4 Test Cases and Checklist for Contract SDKs

Test case	Pass condition
ABI encoding correctness	Byte-for-byte match against a reference encoder across dynamic arrays, tuples, and nested structs
Selector resolution source	Resolved from full verified ABI, not an independently maintained selector table
Malformed calldata decoding	Decoder raises a typed, catchable error; does not crash the host process on truncated or type-mismatched input
Gas estimation safety margin	Includes a documented buffer over the network's bare `eth_estimateGas` response
EIP-1559 fee field support	Correctly sets `maxFeePerGas` / `maxPriorityFeePerGas`; does not silently clamp priority fee to zero
Concurrent nonce handling	Serializes concurrent submissions correctly, or explicitly documents caller responsibility
RPC transport security	Rejects or clearly warns on plaintext HTTP RPC endpoints by default
Chain ID validation before broadcast	Confirms connected chain ID matches expected chain before signing/broadcast is permitted
RPC failover behavior	Fails closed or falls back to a verified secondary endpoint on unreachable/inconsistent primary

8. Front-End and Mobile SDKs

Front-end and mobile SDKs bring the risks already covered in the CDN and Front-End Supply Chain Security Handbook (01) into the SDK's own delivery and runtime, then add the platform-specific storage and sandboxing concerns of iOS and Android.

8.1 Embedded Content and Script Integrity

An SDK distributed as a CDN-hosted script tag, rather than an npm package bundled at build time, inherits every risk detailed in Handbook 01's content-delivery chapters, and testing here means confirming the SDK's own documentation recommends, and its own release process supports, Subresource Integrity (SRI). Fetch the SDK's recommended embed snippet from its own documentation and check for an `integrity` attribute with a SHA-384 or SHA-512 hash pinned to an immutable, versioned URL, not a floating `latest` alias; an SDK vendor that documents `<script src=".../sdk.js">` without an SRI hash is asking every integrator to trust its CDN indefinitely, the exact posture that let the Ledger Connect Kit compromise propagate to every dApp that had embedded it without pinning.

Where the SDK itself loads secondary content at runtime (an iframe for a hosted checkout or KYC flow, a dynamically injected sub-script for a specific feature), verify that content is similarly integrity-checked or served from an origin the SDK's own Content Security Policy guidance explicitly allowlists, rather than an arbitrary, SDK-controlled third-party domain the integrating team has no visibility into.

8.2 Storage and Session Handling

Front-end SDKs frequently cache session tokens, cached balances, or user preferences in browser storage, and the choice of storage mechanism determines the blast radius of a cross-site scripting flaw elsewhere on the same page. Test what the SDK writes to `localStorage`, `sessionStorage`, and cookies, and flag any session token, API key, or key-derivation material stored in `localStorage` specifically, since it is readable by any script running in the same origin, persists indefinitely, and is not subject to the `HttpOnly` and `SameSite` protections available to cookies.

```
// Runtime storage audit: what does the SDK actually persist, and where
const before = { ...localStorage }, beforeCookies = document.cookie;
await sdk.initialize({ apiKey: "test-key" });
await sdk.connectWallet();
const after = { ...localStorage }, afterCookies = document.cookie;

// Diff and inspect every new key for tokens, keys, or session identifiers
Object.keys(after).filter(k => !(k in before))
  .forEach(k => console.log(`New localStorage key: ${k} = ${after[k]}`));
```

For mobile SDKs, the equivalent test targets the platform's local database and shared preferences files rather than browser storage: confirm session material is written to the platform's encrypted, sandboxed storage (iOS Keychain, Android Keystore-backed encrypted storage) rather than a plain SQLite database or `SharedPreferences` file that is trivially readable on a rooted or jail-broken device, or via a backup extraction.

8.3 Platform-Specific (Web, iOS, Android) Considerations

Each platform carries distinct sandboxing guarantees and distinct ways an SDK can undermine them. On the web, verify the SDK does not require disabling the browser's same-origin policy or a restrictive CSP to function, and that any `postMessage` channel it opens (common for iframe-based wallet or checkout integrations) validates the `origin` of incoming messages rather than accepting any origin. On iOS and Android, the OWASP Mobile Application Security Verification Standard (MASVS) provides the structured verification categories to test an SDK against directly: MASVS-STORAGE (does the SDK use platform-provided encrypted storage), MASVS-NETWORK (does it enforce certificate pinning or at minimum TLS validation on every request), MASVS-PLATFORM (does it request only the permissions it documents needing, with no undocumented background location, contacts, or clipboard access), and MASVS-CODE (is the SDK's release build stripped of debug symbols and does it avoid dynamic code loading at runtime).

Platform	Primary risk	Test
Web	Storage exposed to XSS; CDN/ script integrity	localStorage audit (8.2); SRI presence (8.1); CSP compatibility
iOS	Plaintext data in app sandbox or backups	Keychain usage; MASVS-STORAGE compliance; backup exclusion flags on sensitive files
Android	Data exposure via `SharedPreferences`, backups, or clipboard	Keystore-backed encrypted storage; `allowBackup` manifest flag; clipboard access audit
Both mobile platforms	Excessive or undocumented permission requests	Diff requested permissions against SDK documentation; flag any undocumented permission

Clipboard access deserves a specific mention on mobile: an SDK that reads the clipboard without an explicit, documented reason (some wallet SDKs do this to support paste-to-fill address fields) should be tested for whether it clears or avoids caching clipboard contents, since clipboard-monitoring malware that swaps a copied wallet address for an attacker's own is an established mobile attack pattern independent of the SDK itself.

8.4 Test Cases and Checklist

Test case	Pass condition
SRI on CDN-hosted embed snippet	Vendor documentation recommends an `integrity` attribute pinned to an immutable, versioned URL
Sensitive data in `localStorage` / `sessionStorage`	No session tokens, API keys, or key material found in a runtime storage audit
`postMessage` origin validation	Any SDK-opened message channel validates sender origin before processing
iOS: Keychain usage for secrets	Confirmed via static review or documented architecture; no plaintext secrets in app sandbox files
Android: Keystore-backed storage	Confirmed; `allowBackup` disabled for components handling sensitive data, or sensitive files excluded from backup
Permission requests match documentation	Every requested platform permission has a corresponding, documented feature use
Clipboard access, if present	Documented reason exists; clipboard contents are not cached or logged beyond the immediate paste action
CSP compatibility	SDK functions under a nonce-based, `strict-dynamic` CSP without requiring `unsafe-inline` or `unsafe-eval`

Key controls for Part 2

- Run all four testing lanes (static, dynamic, composition, integration) before manual review gates adoption; no single lane is sufficient on its own.
- Require a CycloneDX 1.6 or SPDX 3.0 SBOM, paired with VEX exploitability statements for any Critical/High finding, as a condition of SDK adoption.
- Verify SLSA provenance (minimum Build L2) and Sigstore signatures for every SDK release your CI installs, not only at initial vendor evaluation.
- Pin exact SDK versions with committed lockfiles; mirror security-sensitive SDKs into a private registry under a documented sourcing policy.
- Default wallet SDKs to structured, typed signing (EIP-712 or equivalent) and treat any blind-signing fallback as a flagged exception, not a default path.
- Validate ABI encoding against a reference implementation and enforce chain ID checks before any transaction broadcast.
- Audit front-end and mobile storage for secrets in unencrypted locations; require platform-native secure storage (Keychain, Keystore) for anything session- or key-related.

Part III: Implementation and Operations

Part III moves from methodology to operations: how software development kit (SDK) security testing becomes a permanent, automated stage of the delivery lifecycle instead of a one-time exercise. It covers gating a pipeline on dependency and secret findings, the criteria for vetting a third-party SDK before it ever lands in a lockfile, how to prepare an SDK codebase for external review, and how to run coordinated disclosure once a vulnerability surfaces in code that hundreds of downstream applications already depend on.

9. Integrating SDK Testing into CI/CD

Continuous integration and continuous delivery (CI/CD) is the actual control surface for SDK security, not the audit report that precedes a release. An SDK sits between a protocol and every application that imports it, so a regression that reaches a tagged version does not stay contained to one team: it ships to every consumer who runs `npm install`, `cargo add`, or `pip install` that week. Treating security testing as a manual pre-release checklist means the check runs once, on one commit, while the pipeline runs on every commit forever. The goal of this chapter is to make the pipeline the enforcement point.

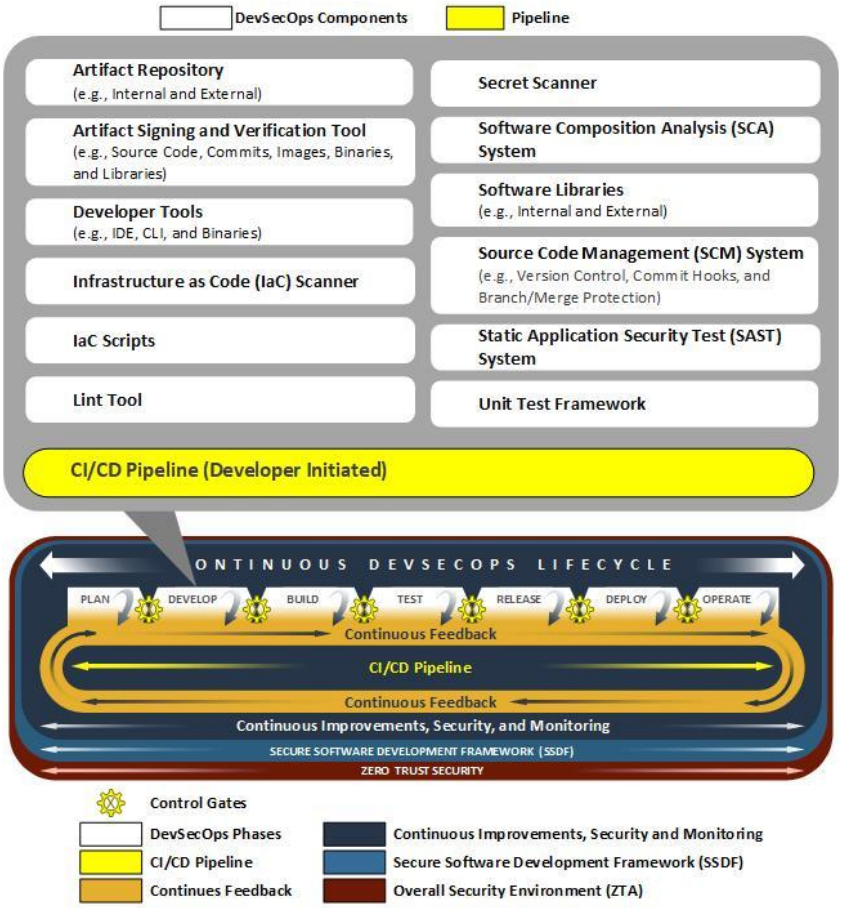

Figure. NIST's Develop phase places security controls beside authoring and review rather than at the release boundary. For SDKs, source review, secret scanning, dependency policy, and secure defaults begin here, before an artifact exists to sign. Source: NIST NCCoE DevSecOps Notational Reference Model, U.S. government work, public domain.

Gate placement. Structure the pipeline as a sequence of gates an artifact must pass before it becomes publishable, not a single "security scan" step appended at the end. Each gate maps to a distinct class of defect and a distinct tool category, and each should be capable of failing the build independently:

Gate	Tool category (examples)	Blocks on	Runs on
Static analysis (SAST)	Semgrep, CodeQL	New high-confidence findings vs. baseline	Every PR
Software composition analysis (SCA)	OSV-Scanner, Trivy, Grype, `npm audit`	New Critical/High advisory in direct or transitive dependency	Every PR and nightly
Software bill of materials (SBOM) generation and diff	Syft, CycloneDX CLI	Undeclared new dependency, license violation	Every merge to release branch

Gate	Tool category (examples)	Blocks on	Runs on
Secret scanning	GitHub secret scanning, Gitleaks, TruffleHog	Any committed credential, API key, or private key pattern	Every push
Provenance and signing	`npm publish --provenance`, Sigstore Cosign	Missing or invalid attestation at publish time	Publish step only

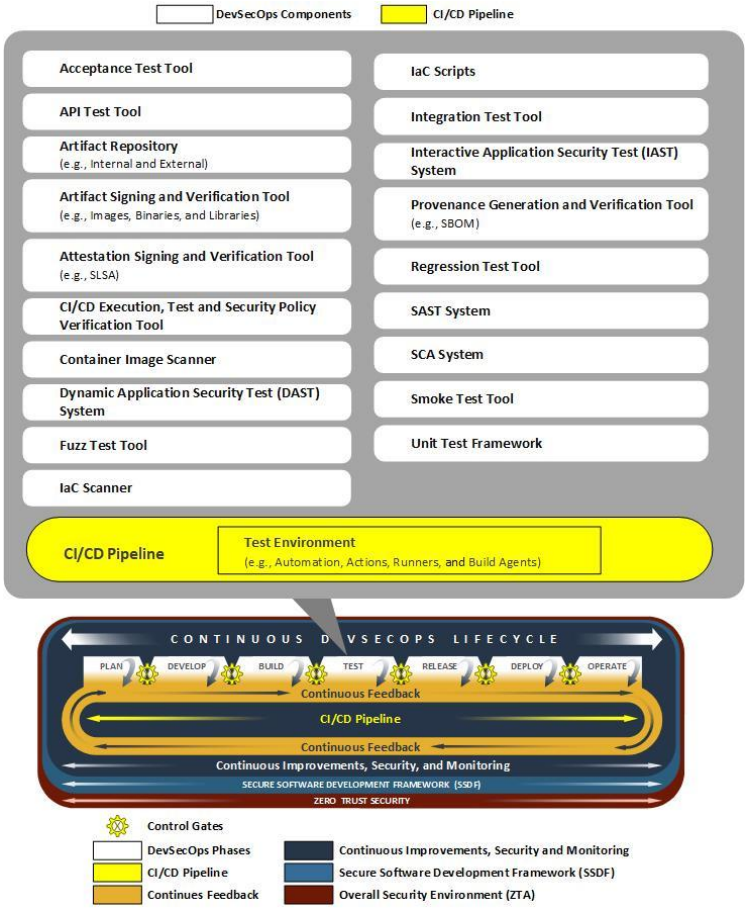

Figure. The Test phase is an evidence-producing gate, not one scanner. Static, dynamic, composition, integration, and manual results converge on an explicit risk decision; failures return the same immutable candidate for remediation rather than being waived invisibly. Source: NIST NCCoE DevSecOps Notational Reference Model, U.S. government work, public domain.

A minimal GitHub Actions job chain that enforces the SCA, SBOM, and secret-scanning gates for an SDK repository looks like this:

```yaml
name: sdk-security-gates
on: [pull_request, push]
jobs:
  sca:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Scan dependencies for known vulnerabilities
        uses: google/osv-scanner-action@v2
        with:
          scan-args: |-
            --lockfile=./package-lock.json
            --fail-on-vuln=true
  sbom:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Generate CycloneDX SBOM
        run: syft dir:. -o cyclonedx-json=sbom.json
      - uses: actions/upload-artifact@v4
        with: { name: sbom, path: sbom.json }
  secrets:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: gitleaks/gitleaks-action@v2
```

Isolate the test environment from the network the way you isolate production. SDK test suites, especially wallet and contract-interaction SDKs, frequently need to talk to an RPC endpoint or a signing device. Do not let CI runners reach live mainnet RPC or hold real credentials during test execution: use recorded fixtures, a local devnet, or a mocked provider, and restrict outbound network egress on the runner to only what the test explicitly declares. This matters for two reasons. It removes test flakiness caused by a third-party RPC outage, and it removes the exact blast radius that a compromised CI runner would otherwise have. The tj-actions/changed-files compromise (CVE-2025-30066) showed what an attacker does with an unrestricted runner: it dumped CI memory to steal secrets exposed in workflow logs across more than 23,000 repositories that referenced the action by a mutable tag.

Publish-time controls close the gap SAST cannot. No static analyzer catches a legitimate maintainer publishing malicious code with valid, currently-authorized credentials, and that is precisely how the field's two most consequential SDK compromises happened (both examined in depth in Chapter 10). The pipeline defense is to remove long-lived publish credentials entirely. npm provenance statements bind a published package to the exact source commit and build workflow that produced it, signed through Sigstore's short-lived certificate chain, so a consumer can verify where a release actually came from. npm Trusted Publishers, an OpenID Connect (OIDC) based mechanism following the same OpenSSF trusted-publishing standard already adopted by PyPI

and RubyGems, goes further: it issues a short-lived, workflow-specific token for each publish instead of a static npm token sitting in a CI secret store, so there is no long-lived credential left to phish.

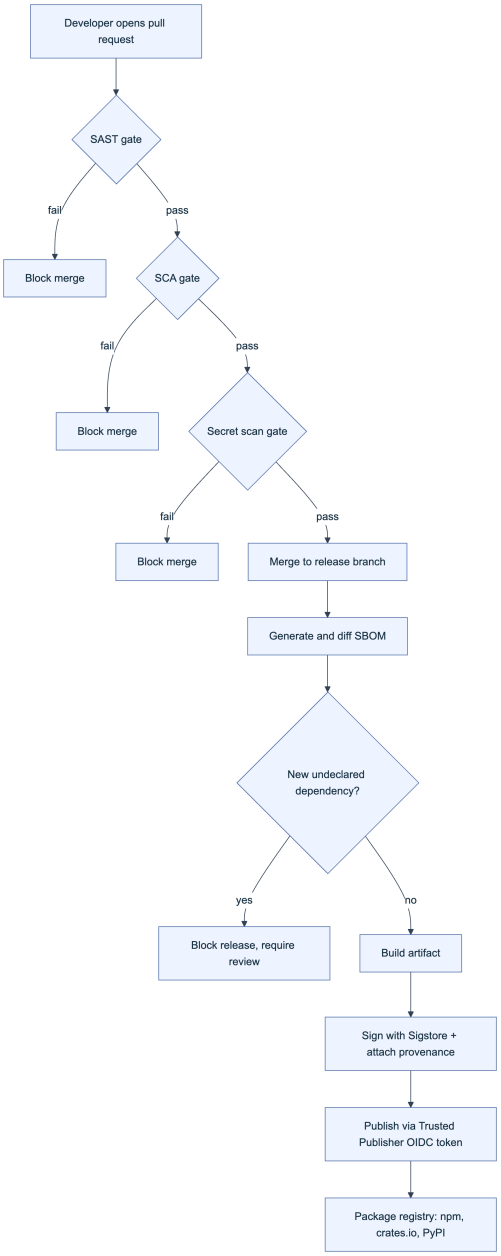

Figure 1. SDK CI/CD pipeline as a chain of independently enforced gates. Each block that can fail the build is a control an attacker who compromises only the CI runner, or only a maintainer's credentials, still cannot bypass alone.

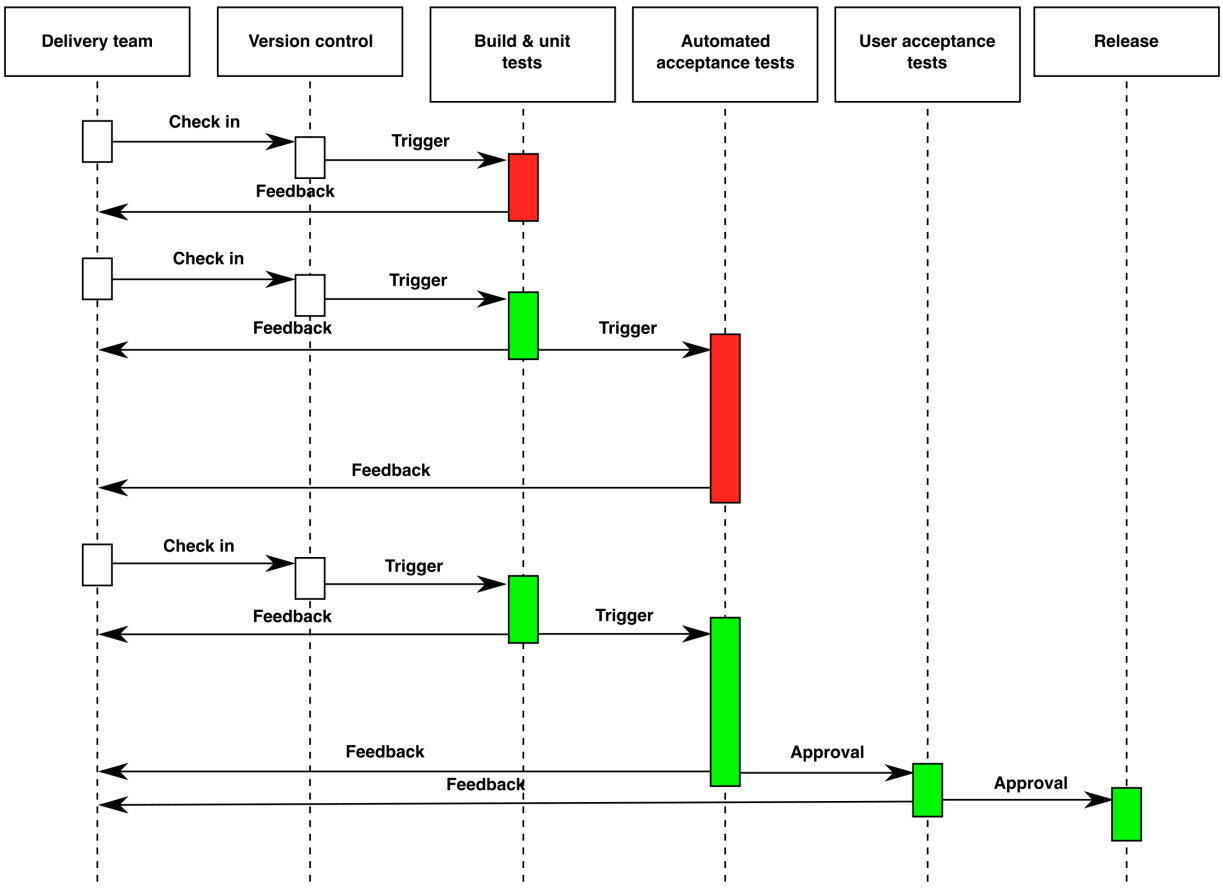

Figure. The general continuous delivery flow that the gate chain above extends with security-specific checks: a change moves through build and automated testing before it becomes a release candidate, and each stage is a point where a pipeline can fail closed. Source: Wikimedia Commons, CC BY-SA 4.0.

10. Selecting and Evaluating Third-Party SDKs

Adopting a third-party SDK is a decision to inherit its entire trust surface. Every dependency it pulls in, every account with publish rights to it, and every incident in its history becomes part of your application's attack surface the moment the import line lands in a lockfile. Popularity and an official-looking name are not evidence of safety: both of the SDK compromises examined below were published under the correct, official package name, by a real maintainer account, and downloaded hundreds of thousands of times before detection.

Evaluate before you adopt, and re-evaluate on every major version bump. A useful evaluation is not a single yes/no gate but a weighted scorecard applied at intake and revisited whenever the SDK's maintainer set, publish process, or dependency tree changes materially.

Criterion	What to check	Why it matters
Publisher provenance	Does the registry entry show npm provenance or an equivalent signed attestation?	Confirms the package matches a specific, auditable source commit, not just a claimed source repository

Criterion	What to check	Why it matters
Maintainer bus factor	How many accounts hold publish rights; is any account without hardware-backed MFA?	A single-maintainer package is one phishing email from compromise
Supply chain hygiene score	OpenSSF Scorecard result: branch protection, pinned dependencies, signed releases, fuzzing/SAST in CI (18 automated checks total)	Gives a repeatable, comparable score instead of an impression
Disclosure policy	Is there a `security.txt` (RFC 9116) or documented vulnerability reporting channel?	Determines whether you can privately report a finding at all
SBOM availability	Does the project publish a machine-readable software bill of materials (CycloneDX or SPDX)?	Lets you diff dependency changes release over release without manually reading every commit
Permission footprint	Does the SDK request network egress, filesystem access, or native code execution beyond what its stated purpose requires?	Bounds the damage of a future compromise of that specific package
Incident history	Has the package, or its maintainers, been involved in a prior compromise?	Past compromise correlates with future targeting; verify remediation, not just the patch version

Case studies: official does not mean uncompromisable. Two incidents from the SDK layer of the Web3 stack illustrate why version pinning with hash verification, not package-name trust, is the load-bearing control.

Date	December 2, 2024	April 21-22, 2025
Malicious versions	1.95.6, 1.95.7	4.2.1-4.2.4, 2.14.2
Attack vector	Publish-access npm account compromised via phishing targeting a maintainer with `@solana` org publish rights (The Hacker News)	Ripple employee's npm credentials compromised via phishing
Malicious payload	Injected `addToQueue` function exfiltrated private keys to `sol-rpc[.]xyz`, funneling funds to a single attacker wallet	Injected `checkValidityOfSeed` function exfiltrated wallet secret key material to an attacker-controlled endpoint
Detection to remediation	Discovered and disclosed within roughly 20 hours by Anza; clean 1.95.8 released same week	Discovered by Aikido Security at 8:14 AM UTC April 22; fully mitigated by 12:34 PM UTC the same day
Tracking ID	CVE-2024-54134	CVE-2025-32965 (CVSS 9.3)
Reported impact	Estimated $130,000-$160,000 in stolen assets	Rotation advised for any key used with affected versions

Both packages were, and remain, the correct official choice for their respective ecosystems. The lesson is not "avoid official SDKs," it is that a package's name and reputation describe its intent, not the current integrity of every byte published under that name. The concrete control this motivates: pin exact versions with lockfile integrity hashes (`package-lock.json` `integrity` fields, `Cargo.lock` checksums, `poetry.lock` hashes), route every dependency bump, including a bump of an already-trusted SDK, through the SCA gate from Chapter 9, and subscribe to the registry's advisory feed (GitHub Security Advisories, `npm audit`, RustSec) rather than relying on manual awareness of vendor incidents.

---

11. Manual Review and External Security Review Preparation

Automated pipeline gates catch what a rule can express. External review supplies an adversarial, unhurried perspective a gate cannot, but only if the codebase handed to reviewers is complete, scoped, and reproducible before the engagement starts. Weak preparation is the most common reason a paid review spends its first days on discovery instead of analysis.

11.1 Documentation, Scope, and Commit Hash; Structural Diagrams

Freeze the review target before reviewers start reading code. Agree an exact commit hash as the fixed point of the engagement, and treat any change to that commit during the review window as a formal scope amendment, not a silent update; reviewers working against a moving target cannot give you a report you can trust against the code you actually ship. Trail of Bits' audit-preparation guidance frames this as four concrete deliverables: state explicit review objectives (what questions should the engagement answer), clean the codebase of dead code, stale branches, and unused dependencies before reviewers arrive, hand over complete build instructions with pinned tool versions, and include the history of prior findings and fixes so reviewers spend time on what has not already been examined.

A documentation package that satisfies this in practice includes:

Deliverable	Content	Why reviewers need it
Scope statement	Exact commit hash, in-scope files/packages, explicitly out-of-scope components	Prevents wasted effort and disputed findings after delivery
Architecture and threat model	Component diagram, trust boundaries, prior STRIDE analysis	Orients reviewers to intended behavior before they look for deviations

Deliverable	Content	Why reviewers need it
Structural diagrams	Data-flow and dependency diagrams for the in-scope surface	Shows where untrusted input enters and where signing or key material lives
Build and test instructions	Exact toolchain versions, reproducible build steps, test coverage report	Lets reviewers reproduce your own test suite before writing new tests
Prior findings	Previous audit reports, unresolved issues, self-identified risks	Directs attention to genuinely unexamined surface

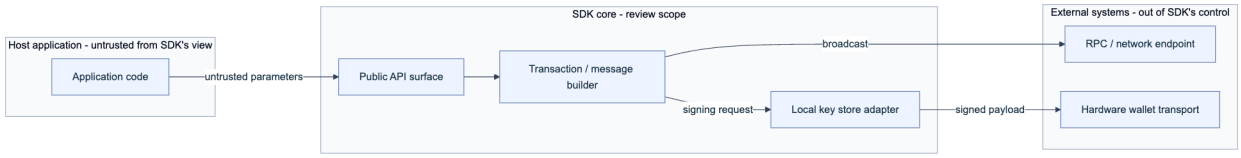

Figure 2. A structural diagram scoped for an external review of a wallet SDK. Marking the trust boundary between the host application, the SDK core, and external systems tells reviewers exactly which inputs are attacker-reachable and where the signing boundary sits.

11.2 Vulnerability Checklists and Offensive Code-Path Analysis

A checklist alone finds the vulnerability classes the checklist's author already anticipated. Pair it with offensive code-path analysis: trace every entry point an attacker (a malicious dependency, a compromised host application, or a hostile RPC response) can reach, follow it forward to the sensitive operation it can influence, and ask what constraint is missing at each hop. This is the same discipline MITRE ATT&CK applies to enterprise intrusions and MITRE AADAPT applies specifically to digital asset and payment technology threats: catalog the technique, then verify the control that should stop it is actually present, rather than assuming it is because the design document says so.

Entry point	Example in an SDK	Sink	Offensive check
Public API parameter	`transfer(amount, recipient)`	Transaction builder	Can a malformed or boundary-value `amount` / `recipient` produce an unintended transaction before signing?
Deserialized RPC response	JSON-RPC result parsed into an SDK object	Balance/ state cache	Can a malicious or MITM'd RPC endpoint inject a value that changes signing logic downstream?
Configuration/plugin loading	Dynamically loaded provider or plugin module	Native code execution	Can an untrusted plugin path escalate to arbitrary code in the host process?
Error and logging paths	Exception messages, debug logs	Log sink, telemetry endpoint	Does an error path leak private key material, seed phrases, or session tokens?
Dependency callback	Third-party library invoked during signing	Signing key access	Does a compromised transitive dependency have a reachable path to key material?

Apply the OWASP SCSTG verification structure as the checklist backbone (input validation, access control, cryptographic handling, error handling), then use the entry-point-to-sink trace above as the offensive layer on top of it. Findings that only a checklist item would catch and findings that only a trace would catch are both real; a review that runs only one of the two methodologies systematically misses the other's blind spot.

12. Responsible Disclosure for SDK Vulnerabilities

An SDK vulnerability has a wider and less controllable blast radius than an application vulnerability, because the SDK is embedded, not deployed: a single fix has to propagate through every downstream application's own release cycle before users are actually protected, and during that gap the vulnerable version keeps shipping to new installs. Disclosure policy for SDK maintainers has to account for that lag explicitly, not just for the maintainer's own patch timeline.

Publish a machine-readable disclosure channel before you need one. A `security.txt` file at `/.well-known/security.txt`, defined by RFC 9116, gives researchers a `Contact` field, an `Expires` timestamp so the file cannot silently go stale, and optionally a `Policy` link and an `Encryption` key for sensitive reports. The OWASP Vulnerability Disclosure Cheat Sheet adds the policy content that file should point to: defined scope, a safe-harbor clause so a researcher testing in good faith is not threatened with legal action, and a stated timeline for acknowledgment, triage, and resolution.

Follow a coordinated vulnerability disclosure (CVD) process, not an ad hoc one. ISO/IEC 29147 defines how a vendor receives and responds to a vulnerability report; the companion ISO/IEC 30111 defines the internal handling process once a report arrives (triage, remediation development, release coordination). For public tracking, request a CVE identifier through a CVE Numbering Authority (CNA); GitHub operates as a CNA for repositories that use GitHub Security Advisories (GHSA), which is how both incidents in Chapter 10 received public CVE IDs alongside their fix.

Timeline benchmarks exist, and web3 SDKs often need something faster. Google Project Zero's disclosure policy is the industry's most cited reference point: a 90-day deadline to public disclosure from initial report, with a 14-day grace period if a fix is imminent, justified by the observation that attackers frequently find the same bugs independently, so silence past a reasonable window protects vendors more than users. That model assumes the bug is *discovered before* exploitation. A key-exfiltrating SDK compromise discovered *during* active exploitation, like both cases in Chapter 10, cannot wait 90 days: the xrpl.js incident went from public detection to a shipped, deprecated-version fix in under five hours precisely because the maintainer, the registry, and the CNA process moved in parallel rather than sequentially. Build your own runbook around two tracks: a standard CVD timeline for vulnerabilities found through review or fuzzing, and an incident-response track (cross-reference the Incident Response Handbook, 06) for anything discovered mid-exploitation, where the first action is pulling the malicious version from the registry, not drafting an advisory.

Phase	Standard CVD target	Active-exploitation target
Acknowledge report	Within 3 business days	Within 1 hour

Phase	Standard CVD target	Active-exploitation target
Confirm and triage	Within 10 business days	Within 4 hours
Registry takedown of malicious version	N/A (no malicious release yet)	Immediate, before further comms
Fix developed and released	Within 90 days (or 90+14 with committed fix date)	Same day where feasible
CVE reserved and published	At or before public disclosure	Concurrent with fix release
Downstream notification (SBOM/VEX consumers, registry advisory feed)	At public disclosure	Concurrent with fix release
Public advisory	30 days after fix, or at 90/104-day deadline	Concurrent with fix release

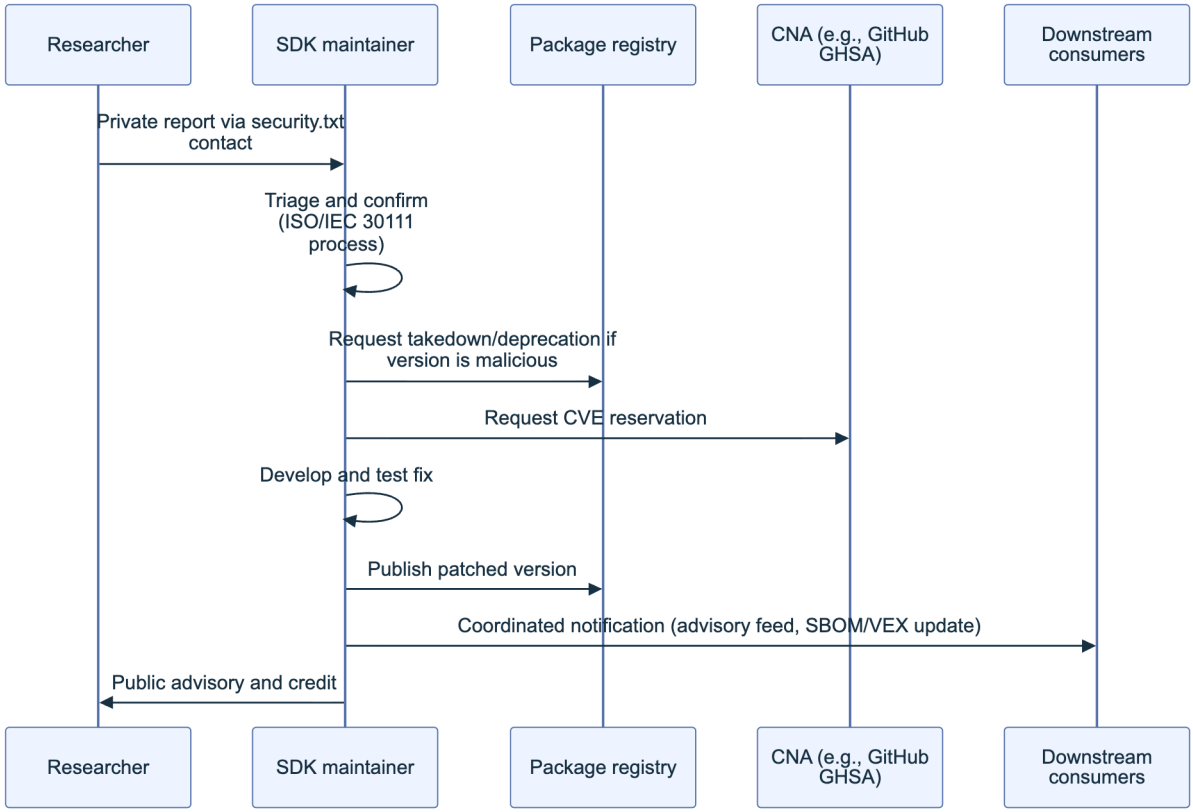

Figure 3. Coordinated vulnerability disclosure flow for an SDK. Registry takedown and downstream notification run in parallel with fix development rather than waiting for it, because the embedded nature of an SDK means every day a malicious or vulnerable version stays installable is a day it keeps reaching new applications.

Use severity classification consumers can act on. Immunefi's framework, used across most web3 bug bounty programs, classifies impact on a five-level scale from Critical to None; adopt an equivalent scale in your own advisory so a downstream application maintainer reading it can immediately judge whether to hotfix or schedule a routine update. Where a bounty program is

warranted for an SDK specifically (as opposed to the protocol it talks to), Immunefi and HackenProof both support SDK- and library-scoped programs distinct from the smart contract bounty, which keeps report volume and severity calibration appropriate to the SDK's actual blast radius.

Key controls for Part 3

- Gate every merge and every publish on SAST, SCA, SBOM diff, and secret scanning; make each gate independently capable of blocking the build.
- Eliminate long-lived publish credentials with npm provenance and OIDC-based trusted publishing; both major SDK compromises examined here were credential compromises, not code defects.
- Score third-party SDKs before adoption (provenance, OpenSSF Scorecard, bus factor, disclosure policy, SBOM availability) and re-score on every major version bump, not just at first intake.
- Freeze review scope to an exact commit hash and hand reviewers a complete documentation package, including structural and trust-boundary diagrams, before the engagement starts.
- Run offensive entry-point-to-sink tracing alongside checklist review; each catches vulnerability classes the other misses.
- Publish a `security.txt` and a written CVD policy before you need one, and maintain a separate, faster runbook for vulnerabilities discovered during active exploitation.

Part IV: Appendices

This part is the handbook's reference layer: a place to look up a term, run a checklist, or trace a citation, rather than a narrative meant to be read start to finish. Appendix A fixes the vocabulary used across Parts 1 through 3 in a single glossary. Appendix B turns nine chapters of testing methodology into copy-ready checklists a reviewer can run against a candidate SDK before it enters a codebase, and Appendix C points to the full bibliography behind every standard and incident cited along the way.

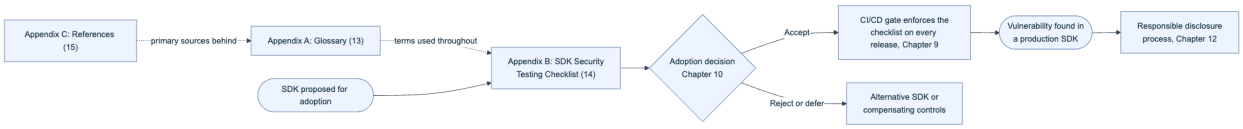

Figure 1. How the appendices connect to the SDK lifecycle described in Parts 1 through 3: vetting feeds the checklist, the checklist gates CI/CD, and a production finding feeds back into disclosure.

13. Appendix A: Glossary

The terms below are defined the way this handbook uses them, not in the generic sense a dictionary would give. Each entry expands its acronym on first mention and, where useful, references back to the section that puts the term into practice, so a reader arriving here from a search result can trace a definition to the place it actually matters rather than stopping at an abstract answer. Alphabetical order is used throughout; where two related terms exist, the more specific one carries the fuller definition and the other cross-references it.

AADAPT. MITRE's Adversarial Actions in Digital Asset Payment Technologies knowledge base, a project structured like ATT&CK but scoped to tactics and techniques specific to cryptocurrency and digital-asset systems. Cited alongside ATT&CK when threat-modeling SDK-facing attack scenarios (2.4).

ABI (Application Binary Interface). The typed description of a smart contract's callable functions, their argument encoding, and their return types, which a contract-interaction SDK consumes to build a transaction's calldata. Malformed or stale ABI handling is a primary defect class covered in Chapter 7 (7.1).

Blind signing. Approving a transaction from a wallet interface's rendered summary without independently verifying the raw calldata that summary claims to describe. A wallet SDK that cannot render a human-readable preview of a nonstandard call forces this on the user by default (6.1, 6.3).

Breaking change. A change to an SDK's public interface, return type, or default behavior that is not backward compatible with the prior minor or patch release, and that therefore requires a major version bump under semantic versioning (5.4).

Calldata. The raw, ABI-encoded byte string a transaction carries to invoke a contract function, distinct from the plain-language summary a wallet's signing interface renders for the user. Contract-interaction SDKs construct it; wallet SDKs are responsible for making it inspectable before signature (7.1, 6.1).

Cosign. Sigstore's command-line tool for signing and verifying software artifacts, container images, and Software Bills of Materials using short-lived, identity-bound certificates instead of a long-lived private key a maintainer has to protect indefinitely (4.4).

CycloneDX. An OWASP-originated Software Bill of Materials standard, now also standardized as ECMA-424, whose 1.6 release in April 2024 added machine-learning-model and cryptographic-asset component types alongside the traditional software inventory. The reference SBOM format used throughout Chapter 4 (4.2).

Dependency confusion. An attack that tricks a package manager into resolving an internal, unpublished package name to a higher-versioned malicious package of the same name on a public registry, demonstrated at scale in Alex Birsan's February 2021 research against more than 35 organizations including Apple, Microsoft, and PayPal. A named risk in both the threat model (2.3) and the sourcing-policy guidance (4.7).

Fulcio. Sigstore's certificate authority, which issues a short-lived code-signing certificate, valid on the order of minutes, bound to a verified OpenID Connect identity rather than a long-term key a signer must safeguard (4.4).

Malicious package. A package published to a public registry that is intentionally harmful, whether typosquatted, dependency-confused, or a previously legitimate package taken over through a compromised maintainer account, the pattern behind the Ledger Connect Kit compromise of December 2023. The primary threat-actor category opened in Chapter 2 (2.2).

MITRE ATT&CK. A globally accessible knowledge base of adversary tactics and techniques whose Enterprise matrix lists Supply Chain Compromise as technique T1195 under Initial Access, with sub-techniques for compromising software dependencies and development tools (T1195.001), the software supply chain broadly (T1195.002), and hardware (T1195.003). The reference framework behind Chapter 2's STRIDE and scenario mapping (2.4).

npm provenance. A feature of the npm registry that lets a publisher attach a Sigstore-backed attestation linking a published package to the exact source commit and public build workflow that produced it, surfaced as a verified badge on the package's registry page. A concrete, freely available instance of SLSA-aligned build provenance (4.4).

OSV (Open Source Vulnerability database). A distributed, machine-readable vulnerability database maintained by Google that aggregates advisories from GitHub Security Advisories, PyPA, RustSec, and other ecosystem-specific sources, queryable by package version or commit hash through its API and the `osv-scanner` command-line tool. A recommended source for continuous Software Composition Analysis (4.5).

Provenance. Verifiable metadata describing how, where, and from what source an artifact was built: which repository, which commit, which build platform, and which workflow produced it. The deliverable every SLSA build level above L0 exists to generate (4.3, 4.4).

Reproducible build. A build process that, given the same source and build environment, produces bit-for-bit identical output every time, letting an independent party rebuild a released artifact and confirm it matches what the vendor published. Distinct from signed provenance, which proves who built something rather than that anyone else can rebuild it identically (4.6).

Rekor. Sigstore's public, append-only transparency log recording every signing event, so an artifact owner can monitor the log for signatures issued under their identity that they never requested (4.4).

RPC (Remote Procedure Call) endpoint. The network interface, typically JSON-RPC over HTTPS or WebSocket, through which a contract-interaction SDK submits transactions and queries chain state. A malicious or misconfigured endpoint can misreport gas prices, censor a transaction, or profile the user's address and activity (7.3).

SBOM (Software Bill of Materials). A structured, machine-readable inventory of every component, direct and transitive, in a piece of software, expressed in a format such as CycloneDX or SPDX. The baseline artifact Chapter 4 asks every SDK consumer to generate and every SDK vendor to publish (4.2).

SCA (Software Composition Analysis). Automated identification of the open-source components inside a codebase and their known vulnerabilities, run continuously rather than as a one-time scan. The testing discipline behind Chapter 3's composition analysis and Chapter 4's continuous-scanning guidance (3.2, 4.5).

SCSVS / SCSTG / SCWE. The Smart Contract Security Verification Standard, the Smart Contract Security Testing Guide, and the Smart Contract Weakness Enumeration, the three OWASP Smart Contract Security Project standards this handbook extends to the SDK layer (1.3).

Semantic versioning (semver). A version-numbering scheme, `MAJOR.MINOR.PATCH`, in which a major bump signals a breaking change, a minor bump signals backward-compatible new functionality, and a patch signals a backward-compatible fix. The convention Chapter 5 expects an SDK to honor before a consumer trusts a caret or tilde version range (5.4).

Sigstore. A Linux Foundation project providing keyless signing and verification for software artifacts through three components: Fulcio (certificate authority), Rekor (transparency log), and Cosign (signing and verification client) (4.4).

SLSA (Supply-chain Levels for Software Artifacts). The current v1.2 specification defines two independently assessed tracks: Build L0–L3 for provenance integrity and build-platform isolation, and Source L0–L4 for version control, history/provenance, continuous technical controls, and two-party review. A claim must name its track and level; "SLSA Level 3" alone is ambiguous (2.6, 4.3).

SPDX. The System Package Data Exchange format, an ISO/IEC 5962:2021 international standard for expressing SBOMs. SPDX 3.0, released in April 2024, moved to an RDF-based data model with eight specialized profiles (Core, Software, Security, Build, AI, Dataset, Licensing, Lite), replacing the flatter document structure of SPDX 2.x (4.2).

STRIDE. A threat-modeling mnemonic, Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege, applied to SDK-specific attack scenarios in Chapter 2 (2.4).

Trust boundary. A point in a system where data or control passes from one level of trust to another: from an SDK's internal state into the calling application, or from a remote registry into a local build. Enumerating every trust boundary an SDK crosses is the first step of Chapter 2's threat model (2.1).

Typosquatting. Publishing a malicious package under a name that is a common misspelling or visual near-match of a popular package, betting that a developer's typo or a copy-paste error installs the wrong one. A named registry-level threat in Chapter 2 (2.2, 2.3).

UI redress. An attack that visually or structurally disguises a malicious interface element as a legitimate one, including clickjacking, to trick a user into approving an action, such as a wallet-connect request, they did not intend. Treated as a wallet-SDK risk distinct from pure credential phishing (6.3).

VEX (Vulnerability Exploitability eXchange). A companion document to an SBOM stating whether a known vulnerability in a listed component is actually exploitable in the product's specific configuration, using status values including affected, not affected, fixed, and under investigation, as defined in CISA's minimum requirements for VEX. Lets an SDK vendor tell consumers which CVEs in their dependency tree actually matter (4.2).

Version pinning. Locking a dependency to an exact version or exact content hash rather than a floating range, so a build resolves to identical code every time until a human deliberately updates it. The default sourcing policy Chapter 4 recommends for any SDK with signing or key-handling responsibility (4.7).

---

14. Appendix B: SDK Security Testing Checklist

This checklist consolidates Parts 2 and 3 into one printable document a reviewer can run against a specific SDK, at a specific version, before it is adopted and again at every subsequent release. Run one full copy at first adoption and the "Supply Chain and Dependency Testing" and "CI/CD Gate" groups on every version bump after that; a checklist that is completed once at adoption and never rerun does not catch a maintainer takeover that happens eighteen months later, which is exactly the shape the Ledger Connect Kit and Polyfill.io compromises took. Mark any inapplicable line N/A with a stated reason rather than leaving it blank, since an empty line and a deliberately skipped one look identical to a later reviewer.

Assign a risk tier before applying a minimum bar. An SDK that touches private key material carries different consequences from an SDK that only renders a marketing banner, and a single blanket requirement either overburdens the second case or underprotects the first.

SDK record

- SDK name and version under review: _____
- Registry / source URL: _____
- Risk tier (see table below): _____
- Reviewer: _____ Date: _____
- Adoption decision: Accept ☐ Accept with compensating controls ☐ Reject ☐

Risk tier	Example SDK type	Minimum SLSA build level	Signing / SBOM bar	Review cadence
Tier 1: Key custody	Wallet SDK, signing library	L3	Sigstore-signed release plus SBOM with VEX	Every release, plus a quarterly re-vet regardless of new releases
Tier 2: Value-moving	Contract-interaction SDK, RPC client	L2 minimum, L3 preferred	Signed release plus SBOM	Every release
Tier 3: Rendering / UX	Front-end embed, mobile UI kit	L1 minimum	SBOM required, signing preferred	Every minor version
Tier 4: Non-critical utility	Logging or analytics wrapper	L0 acceptable if SRI and CSP compensating controls are in place	SBOM optional	Annual

SDK adoption and vetting (Chapter 10)

- ☐ Maintainer identity and publishing history reviewed; no anonymous or freshly created account holds publish rights
- ☐ Package ownership and maintainer-transfer history checked for an unexplained handoff

- ☐ Risk tier assigned from the table above and the matching minimum SLSA level confirmed against the vendor's actual release (4.3)
- ☐ License terms confirmed compatible for the SDK and its transitive dependencies
- ☐ At least one alternative SDK compared, and the adoption decision and rationale recorded

Supply chain and dependency testing (Chapter 4)

- ☐ Full dependency tree resolved, direct and transitive, and diffed against the SBOM the vendor publishes (4.1, 4.2)
- ☐ SBOM generated or obtained in CycloneDX or SPDX format; VEX statements reviewed for every known CVE in the tree (4.2)
- ☐ Build provenance verified against the claimed SLSA level: signature present, builder identity matches, source repository matches (4.3, 4.4)
- ☐ Sigstore (or equivalent) signature verified before first use and again on every update (4.4)
- ☐ Dependency tree scanned against OSV or an equivalent database; no unresolved Critical or High severity findings (4.5)
- ☐ Build reproducibility spot-checked where the project supports it (4.6)
- ☐ Every dependency pinned to an exact version or content hash; no floating range survives in the lockfile (4.7)
- ☐ Registry source confirmed as the intended public or private registry, not an unofficial mirror (4.7)

API and interface security (Chapter 5)

- ☐ Every SDK input, addresses, amounts, ABI fragments, callback URLs, validated and sanitized before use (5.1)
- ☐ No API key, private key, or secret accepted as a plain constructor argument or written to a log in cleartext (5.2)
- ☐ Error messages reviewed for stack traces, internal file paths, or secret material leaking to the caller or console (5.3)
- ☐ Declared version range tested against the actually installed version for undocumented breaking-change drift (5.4)

Wallet and key-handling SDKs (Chapter 6)

- ☐ Key generation uses a cryptographically secure random source, verified against the platform's documented CSPRNG (6.1)
- ☐ Private key material never leaves the device or hardware wallet unencrypted (6.1, 6.2)
- ☐ Hardware and external wallet connectors (WalletConnect, browser-extension bridge) verified against the vendor's current, signed release (6.2)
- ☐ Transaction and message previews render enough decoded calldata that blind signing is not the only option available to the user (6.1, 6.3)

- [ ] Connection and approval prompts tested for UI redress and origin-spoofing resistance (6.3)

Contract interaction SDKs (Chapter 7)

- [ ] ABI encoding and decoding tested against boundary cases: zero values, maximum uint, empty arrays, malformed fragments (7.1)
- [ ] Gas estimation tested under simulated network congestion and against a stale gas oracle (7.2)
- [ ] RPC failover and response validation tested; the SDK does not silently trust an unverified single RPC response (7.3)
- [ ] Chain ID and network context checked before every transaction to prevent cross-chain replay (7.3)

Front-end and mobile SDKs (Chapter 8)

- [ ] Any script or asset the SDK loads at runtime is served from an immutable URL and pinned with Subresource Integrity where the platform supports it (8.1)
- [ ] Local storage, session storage, and mobile keychain/keystore usage reviewed for sensitive data held at rest (8.2)
- [ ] Platform-specific permissions (iOS Keychain access groups, Android Keystore, browser storage partitioning) reviewed against least privilege (8.3)

CI/CD gate (Chapter 9)

- [ ] SBOM generation, signature verification, and dependency scanning run as blocking pipeline steps, not advisory ones (9)
- [ ] A version or hash bump in a pinned SDK dependency requires an explicit, reviewed pull request, never an automatic floating update (9, 4.7)
- [ ] Pipeline fails closed: a scan or verification tool timeout blocks the merge rather than defaulting to pass (9)

Pre-external-review preparation (Chapter 11)

- [ ] Scope document lists the exact commit hash, files, and SDK version under review (11.1)
- [ ] Architecture and data-flow diagrams updated to match the commit under review (11.1)
- [ ] Known-issue and out-of-scope list prepared so reviewer time is not spent re-discovering accepted risk (11.2)

Responsible disclosure (Chapter 12)

- [ ] Security contact and disclosure policy published and current, for example a `security.txt` file per RFC 9116 (12)

- ☐ Triage and emergency patch-release process rehearsed with a dry run before it is needed for real (12)
- ☐ Downstream consumer notification plan defined for a confirmed SDK-level vulnerability (12)

Once the CI/CD gate is wired in, the pipeline should enforce the supply chain group mechanically rather than depending on a human to remember it on every dependency bump:

```
# Minimum verification bundle before a new SDK version enters the dependency tree.
        {.after-printable-checklist}
# Every step below is a blocking check, not an advisory one (Chapter 9).

npm audit signatures                                          # confirm npm provenance/re-
        gistry signatures

cosign verify-blob \
  --certificate-identity-regexp '^https://github.com/ORG/REPO/' \
  --certificate-oidc-issuer 'https://token.actions.githubusercontent.com' \
  --signature sdk-release.sig sdk-release.tgz              # verify Sigstore signature
        and builder identity

osv-scanner --lockfile package-lock.json                  # scan the resolved depend-
        ency tree against OSV

syft packages sdk-release.tgz -o cyclonedx-json > sbom.json  # generate an SBOM if the
        vendor did not publish one
```

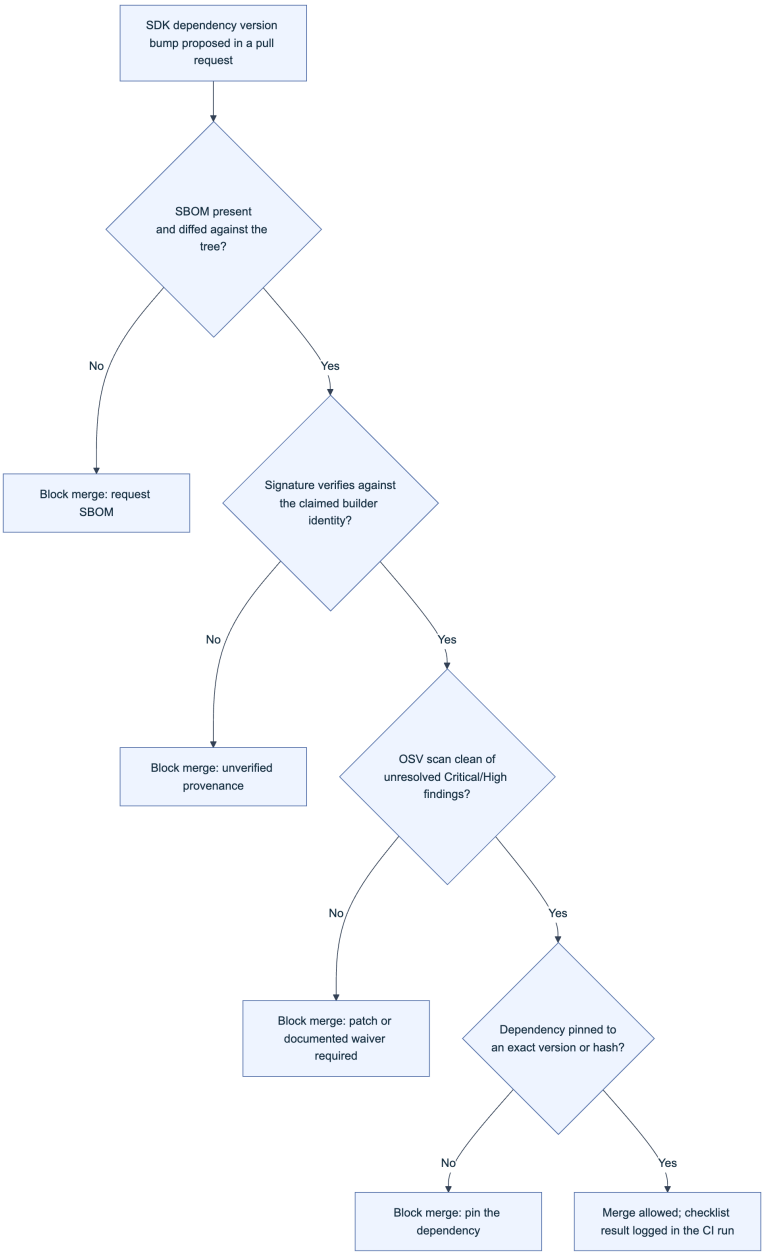

Figure 2. The CI/CD gate from Chapter 9 expressed as a fail-closed decision sequence. Every "No" branch blocks the merge; nothing in this chain is advisory, matching the checklist item above it.

---

15. Appendix C: References and Further Reading

The full, deduplicated bibliography for this handbook lives in a separate file, `references.md`, kept apart from the narrative parts so the source list can be maintained and re-verified on its own schedule. Every inline citation across Parts 1 through 3, and in Appendix A above, points to a primary source: a standard's own specification page, a vendor's or researcher's incident disclosure, or a named tool's documentation, never a secondary summary of one. `references.md` groups those sources so a reader chasing one kind of citation, a standard's version history, or a specific incident's timeline, does not have to re-scan three parts of prose to find it.

Category	What it holds	Representative sources
Standards and Frameworks	The specifications and control frameworks this handbook's testing methodology traces back to	SLSA v1.2 Build and Source Tracks, CycloneDX 1.6 / ECMA-424, SPDX 3.0, Sigstore (Fulcio, Rekor, Cosign), OWASP Top 10:2025 (A03 Software Supply Chain Failures), SCSVS, SCSTG, MITRE ATT&CK T1195, MITRE AADAPT, RFC 9116
Incidents and Case Studies	Named SDK and supply chain compromises cited as evidence for a control	Ledger Connect Kit (December 2023), Polyfill.io (June 2024), tj-actions/changed-files CVE-2025-30066 (March 2025), the XZ Utils backdoor (2024), Alex Birsan's dependency confusion disclosures (2021)
Tools	Vendor and project documentation for named tools referenced in the checklist and CLI examples	Cosign, OSV-Scanner, Syft, npm provenance documentation, GitHub Security Advisories
Further Reading	Cross-references to companion OWASP SCS handbooks	CDN and Front-End Supply Chain Security (01), Incident Response (06), Web3 Attack Vectors Mapping (10)

Treat `references.md` as the place to verify a claim before repeating it outside this handbook, particularly an incident's figures: public reporting on an active or recent compromise is routinely revised as forensic detail matures, and a number cited here reflects what was known at the time of writing.

Key controls for Part 4

- Keep Appendix A within reach for any reader, new to Web3 SDK security or not, who lands on a chapter mid-stream and needs a term defined in context rather than in the abstract.
- Run one full Appendix B checklist at SDK adoption, and rerun the supply chain and CI/CD groups at every subsequent release, not only at the first integration.
- Assign a risk tier before applying a minimum SLSA level or signing bar; a key-handling SDK and a logging wrapper do not warrant the same gate.
- Verify signatures and SBOMs with the CLI bundle in Appendix B directly, rather than trusting a badge or claim on a vendor's landing page.
- Route to `references.md` before repeating an incident's figures externally, since active investigations are revised after initial disclosure.

This is the final part of the OWASP SCS SDK Security Testing Handbook.

Appendix C: References and Further Reading

The sources below are cited throughout this handbook and are grouped here for quick reference.

Standards and Frameworks

- MITRE AADAPT
- MITRE ATT&CK
- MITRE ATT&CK - Technique T1195: Supply Chain Compromise
- OWASP Vulnerability Disclosure Cheat Sheet
- CycloneDX 1.6 JSON Specification
- EIP-1193: Ethereum Provider JavaScript API
- EIP-1559: Fee Market Change for ETH 1.0 Chain
- EIP-3085: Wallet Add Ethereum Chain RPC Method
- EIP-712: Typed Structured Data Hashing and Signing
- OpenVEX Specification
- OWASP Mobile Application Security Verification Standard (MASVS)
- OWASP Top 10:2025
- OWASP Smart Contract Security Testing Guide / SCSVS / SCWE (scs.owasp.org)
- OWASP Web3 Attack Vectors Top 15
- SLSA v1.2 Specification
- SLSA v1.2 - Build Requirements
- SLSA v1.2 - Source Requirements
- SLSA v1.2 - Threats and Mitigations
- SLSA v1.2 - What's New
- SPDX 3.0 Specification
- CISA VEX Working Group
- CISA, Minimum Requirements for Vulnerability Exploitability eXchange (VEX)
- ISO/IEC 30111: Vulnerability Handling Processes
- ISO/IEC 29147: Vulnerability Disclosure
- RFC 9116: A File Format to Aid in Security Vulnerability Disclosure (security.txt)

Incidents and Case Studies

- XcodeGhost (Wikipedia)
- event-stream Malicious Dependency Incident - GitHub Issue #116

- Alex Birsan - Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies (2021)
- Doctor Web - Malicious npm Package Analysis
- Socket - npm Author “Qix” Compromised in Major Supply Chain Attack
- Socket.dev - Supply Chain Attack on the @solana/web3.js Library
- The Hacker News - Crypto Trading Firm Wintermute Loses $160M in Hack
- The Hacker News - Researchers Uncover Backdoor in Solana’s Web3.js npm Library
- Aikido Security - npm debug and chalk Packages Compromised
- Aikido Security - XRP Supply Chain Attack: Official npm Package Infected with Crypto-Stealing Backdoor
- CISA Alert - Supply Chain Compromise of Third-Party tj-actions/changed-files (CVE-2025-30066)
- Halborn - Explained: The Profanity Address Generator Hack (September 2022)
- Ledger - A Letter from Ledger Chairman & CEO Pascal Gauthier Regarding the Ledger Connect Kit Exploit
- NCC Group - In-Depth Technical Analysis of the Bybit Hack
- XRPL.org - Vulnerability Disclosure Report: xrpl.js Bug (April 2025)

Tools and Documentation

- npm Documentation - Generating Provenance Statements
- npm Documentation - Trusted Publishers
- Sigstore Documentation (Cosign, Fulcio, Rekor)
- GitHub Security Advisories (GHSA) Database
- National Vulnerability Database (NVD)
- OSV.dev - Open Source Vulnerabilities Database
- Reproducible Builds Project
- OpenSSF Scorecard
- Sourcify - Contract Source Verification
- WalletConnect Network

Further Reading

- Trail of Bits - How to Prepare for a Security Audit
- Immunefi - Learn (Bug Bounty Severity Classification Framework)
- Google Project Zero - Vulnerability Disclosure FAQ